



Reducing cross-validation variance through seed blocking in hyperparameter tuning

Giovanni Maria Merola¹

Received: 14 August 2023 / Revised: 5 December 2024 / Accepted: 12 December 2024 /

Published online: 17 February 2025

© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2025

Abstract

Hyperparameter tuning plays a crucial role in optimizing the performance of predictive learners. Cross-validation (CV) is a widely adopted technique for estimating the error of different hyperparameter settings. Repeated cross-validation (RCV) is commonly employed to reduce the variability of CV errors. This study investigates the efficacy of blocking cross-validation partitions and algorithm initialization seeds during hyperparameter tuning. The proposed approach, termed Controlled Cross-Validation (CCV), reduces variability in error estimates, enabling fairer and more reliable comparisons of predictive model performance. We provide both theoretical and empirical evidence to demonstrate that this blocking approach lowers the variance of the estimates compared to RCV. Our experiments indicate that the algorithm's internal random behavior often does not significantly affect CV error variability. We present extensive examples using real-world datasets to compare the effectiveness and efficiency of blocking the CV partitions when tuning the hyperparameters of different supervised predictive learning algorithms.

Keywords Repeated CV · Controlled cross-validation · Cross-validation standard error · Random seed · Random behavior · Nonparametric tests

1 Introduction

The performance of a machine learning algorithm, or learner,¹ on a given learning task depends on its hyperparameter settings. Hyperparameters are parameters of the algorithm that cannot be estimated from the data but need to be set before the algorithm is run. A setting is a combination of hyperparameter values under which the algorithm is run. Tuning

¹ We refer to algorithms that use a set of features to predict a response generically as “predictive learners” or simply “learners”. We reserve the term “model” for a theoretical description of the relationship among variables.

Editor: Derek Greene.

✉ Giovanni Maria Merola
giovanni.merola@buv.edu.vn

¹ School of Business, British University Vietnam, Hung Yean, Vietnam

the hyperparameters of a learner is crucial, as it involves choosing the best settings to maximize the learner's performance on a given dataset.

K-fold cross-validation (CV, Stone Allen 1974 and 1974 is a simple, widely used method for measuring the predictive power of a supervised predictive learner, often used for tuning the hyperparameters $\theta = (\theta_1, \theta_2 \dots)$ of a supervised predictive learner. CV consists of partitioning the dataset into a predetermined number of *folds*, usually five or ten. Each fold is used as a validation sample for the learner trained on the remaining folds, and the CV prediction error (CVerr) is computed as the average of the resulting prediction errors. The popularity of CV mostly comes from the “universality of the data splitting heuristics” (Arlot & Celisse, 2010).

CV is also used to estimate the properties of the prediction error of future, yet to be seen, data. When used for hyperparameter tuning, the CVerr serves only as a measure of a learner's predictive accuracy, irrespective of what it estimates. Therefore, we focus on the properties of CVerr when used for tuning learners within the available training data. Discussion about the estimation of the future error using CV can be found, for example, in Arlot and Celisse (2010) and Bates et al. (2021) and references therein.

Hyperparameter tuning is data-dependent, requiring specific adjustments for each dataset. This process is challenging because the space of feasible hyperparameter settings is potentially enormous, making it computationally demanding. Expert knowledge is often necessary to select which parameters to tune and their range (e.g., Probst et al., 2019). Various approaches have been suggested to limit the search space, improve computational efficiency, or automate the process (Bischl et al., 2023; Luo, 2016; Yang and Shami, 2020, among others)

One of the concerns about using CV for choosing settings is that the CVerr values for the same setting will vary if computed on different CV partitions. Consequently, when comparing settings, one setting might yield a lower CVerr than another simply because it was computed on a CV partition more favorable to predictions, whereas on many other partitions it would have resulted in a higher CVerr.

While repeated cross-validation (RCV) has been widely adopted to mitigate variability, this study proposes Controlled Cross-Validation (CCV) as a more effective alternative. By systematically blocking cross-validation partitions and algorithm seeds, CCV achieves significant reductions in error variability, as demonstrated both theoretically and empirically.

In this paper, we address the problem of comparing the CVerr computed for different settings and different CV partitions. To do so we consider the simplest and most popular hyperparameter tuning method, *grid search* (GS). In GS, the CVerr is computed for all combinations of preselected hyperparameter values, and the setting that corresponds to the lowest error is chosen as the most promising. Due to the limited number of settings considered, this discrete search is unlikely to find the optimal setting, but it is computationally feasible and expected to identify good settings.

The simplicity of GS makes it easier to explain and exemplify the results of our research. However, the principle of comparing the CVerr values computed on the same CV partition that we will explore is applicable to most tuning approaches that use performance measures based on CV.

Another source of variation in the CVerr can be observed in algorithms that rely on random decisions, such as resampling, variable selection for individual trees, or random dropouts. These decisions can produce different values of the CVerr for the same CV partition unless the random behavior is fixed by using the same initial random seed. Consequently, suboptimal settings may be selected during grid searches due to the use of CVerr values computed on varying CV partitions and initial seeds. CV partitions are typically generated

by splitting the observations with a single (pseudo)random number generator. To simplify terminology and avoid unnecessary complexity, throughout this paper, we refer to different CV partitions by the initial seed used to create them, denoted as the CVseed. Similarly, we generically refer to the initial seed for an algorithm's random behavior as the RBseed.

It is important to note that CV partitions and random behavior are the only sources of variation in CVerr for a fixed setting. The variation due to the CV partition is not part of a learner's behavior and will not affect future predictions because these will be obtained from the learner trained on the entire training dataset. Therefore, it is crucial to eliminate or, at least, reduce this variability to make fair comparisons of the CVerr values computed under different settings.

Random behavior is more comparable to a random hyperparameter, as the learner used in production will be trained on a single initial seed. However, tuning the initial seed is impractical due to its vast range of possible values and the lack of any meaningful relationship between consecutive seed values, rendering a grid search approach ineffective. Since the starting seed is generally believed to have minimal impact on the learner's predictive performance, treating it as a nuisance variable to reduce computational costs is a reasonable approach.

There are instances where different initial seeds significantly affect algorithm outcomes. For example, Henderson et al. (2018) and Islam et al. (2017) highlight this issue in Reinforcement Learning. However, they advise against searching for the "best" random seed and instead emphasize averaging results over multiple runs, each with a different random seed. We will demonstrate that this variability is also relevant for predictive learners, such as Boosted Trees.

When only one CVerr is computed for each setting in the grid, the CVerr variability due to CV partitions cannot be neither estimated nor controlled. The most sensible approach seems to be to compute all CVerr values on the same partition. We believe this should be standard practice because using different partitions offers no advantage for comparing CVerr values. The variability due to the seeds can only be accounted for if the CVerr is computed more than once for each setting.

One popular method for mitigating the impact of comparing the CVerr values computed on different CV partitions is Repeated Cross-Validation (RCV). In RCV, CVerr is computed on multiple randomly chosen CV partitions for each setting in the grid. The average CVerr for each setting is then compared instead of single values. This process aims to reduce the influence of particularly favorable or unfavorable CV partitions by averaging CVerr values obtained from less favorable or unfavorable ones. This approach leverages the well-known property that the mean has lower variance than individual observations.

While RCV enhances the precision of the measured CVerr, it does not compare the CVerr values of different settings computed on the same CV partitions. Additionally, it should be noted that with this procedure, the initial seed is randomized, and in most cases, the initial seed differs for each CV fold attempted.

In this paper, we explore the effect of measuring CVerr repeatedly for every setting in the chosen grid, each time using the same CV partition and possibly the same algorithm's initial seed. Like in RCV, CVerr values are computed several times for each setting in the grid. However, in each round of repetition, CVerr is computed on the same CV partition for all settings. This allows the CVerr for different settings to be compared on the same CV partition, eliminating the effect of favorable or unfavorable CV partitions on the comparisons, as the effect will be consistent for all settings tried.

Controlling for the CV partition or the initial seed in GS is referred to as *blocking a nuisance variable* in experimental design jargon (for example, Montgomery, 2008). Nuisance

variables are variables that are of no interest but increase response variability. Repeating the experiments while allowing the nuisance variables to take random values, as in RCV, is called randomization. Blocking is far more effective than randomization at reducing the variance of the estimates, as noted by several highly respected sources (Box et al., 2005; Cox and Reid, 2000; Fisher, 1937; Montgomery, 2008; Wu and Hamada, 2011, among others). “Block what you can, randomize what you cannot,” says an old adage in statistics.

In addition to enabling the comparison of the responses of experiments to the same value of the nuisance variable, blocking allows the experimental variance between blocks to be isolated and removed, which cannot be done with randomization. This results in estimated responses with lower standard error, thereby increasing the precision of the estimates.

We will refer to the blocking of the search grid with respect to CVseeds and, when done, RBseeds as Controlled CV (CCV) rather than the more suggestive Blocked CV, because the latter term is already used when CV is applied to time-correlated observations. We will refer to different blockings of the grid as *CV designs*. For example, RCV denotes a design without blocking and CCV4x4 denotes a design with four blocks of two initial seeds (the first will always be the CVseed).

The final effect of CCV and RCV is to reduce the variability of the estimated prediction error for each setting tried. In other words, they reduce the standard error of the mean CVerr (CVstderr) obtained after averaging over the blocks or repetitions. Hence in the following we will use the CVstderr as a measure of performance of different CV designs.

We analyze the CVerr obtained from a hyperparameter grid as a factorial experimental design and analyze it using a linear model Analysis of Variance model (ANOVA) (Montgomery, 2008, among others). ANOVA decomposes the variance of the response into the sum of the variance due to the experimental and error terms. In this analysis, CVseed and RBseed will act as nuisance variables with random effects. By studying the ensuing variance decomposition, we theoretically demonstrate why the CVstderr obtained with CCV is expected to be lower than that obtained with RCV. We will not analyze the effects of the individual hyperparameters because we are interested only in studying the effect of blocking on the variance of the computed CVerr. The analysis of hyperparameters’ effects requires ad-hoc methodologies, briefly mentioned in Sect. 2. Therefore we will treat settings as categorical variables in the ANOVA analysis.

We also provide examples using datasets of small to medium size, comparing characteristics of the CVerr computed by CCV and RCV on three different predictive learners: K-nearest Neighbors, Random Forest, and Gradient Boosted Trees. Since classic F-tests are not appropriate for the analysis of CVerr experiments (Dietterich, 1998), we use permutation tests as guidelines to verify whether factor effects are significantly different from zero. Because the examples are run on a necessarily limited number of blocks or repetitions, they should not be considered empirical proof of the superiority of CCV over RCV, which is given theoretically in Sect. 3.

1.1 Manuscript roadmap

Section 2 provides an overview of the relevant literature. In Sect. 3, we present the theoretical justification for favoring CCV over RCV in parameter tuning and introduce a statistical model for RCV, which, to our knowledge, has not been discussed in prior literature. Section 4 details the results obtained from analyzing six real datasets. In Sect. 5, we

discuss our findings and provide additional considerations. Finally, Sect. 6 offers concluding remarks.

The Appendix includes references for the datasets used in the examples, additional statistics for the numerical analyses not covered in the main body, and a list of the software frameworks reviewed.

2 Related work

Since CV is the most frequently used method for evaluating the predictive accuracy of supervised learners, there is a large body of literature on the CVerr, covering different aspects of CV. Here, we synthesize a few results of interest for our discussion on hyperparameter tuning.

The origin of RCV is uncertain but it is a widely used technique in applications and textbooks. We could not find theoretical works that analyze the effect of repetitions on the CVerr when used for hyperparameter tuning. However, the increase in precision due to randomization is recognized also in the context of estimation (for example, Kim, 2009; Molinaro et al., 2005).

To the best of our knowledge, there are no theoretical or empirical works that explicitly analyze controlling for CV partitioning or algorithm's random behavior to reduce the CVerr variance when tuning a learner. Additionally, we found no discussion of a statistical model that includes the variance due to CV partitioning or random behavior in hyperparameter tuning.

The concept of controlling for CV partitions in hyperparameter tuning has appeared in the literature a few times. Dietterich (1998) discusses the need to compare the estimated cross-validation error of different learning algorithms computed on the same CV partitions. In describing the ParamILS automatic algorithm optimization framework, Hutter et al. (2009) explains that parameter settings are compared “on exactly the same instances and seeds,” but do not delve into the concept of blocking and use nonparametric paired tests. Later, Alpaydin (2020, p.522) explicitly states that results obtained by resampling must be compared by blocking the subsamples used and suggests using ANOVA to analyze experimental outcomes. Despite the intuitive sense of comparing the CVerr values computed on the same CV partition, these are the only references to this practice that we found in the literature, even in recent articles discussing modified search algorithms such as random, local, or adaptive searches.

A few works treat GS as an experimental design Adenso-Díaz and Laguna (2006), Lujan-Moreno et al. (2018) and Hutter et al. (2014) proposed using specific factorial designs rather than a simple grid, and suggested adaptive local search, response surface methodology and functional ANOVA, respectively, to find optimal settings. Neither, however, mentions controlling for the CV partitioning. Moreover, in extensive recent reviews of hyperparameter optimization methods, Luo (2016), Feurer and Hutter (2019), Yu and Zhu (2020), Yang and Shami (2020) and Bischl et al. (2023) there is no mention of comparing the CVerr values computed on the same partitions, nor of controlling for this or for random behavior in any other way, whether for GS or more sophisticated optimal search approaches. In some cases RCV is considered marginally.

Controlling for cross-validation partitions or random behavior when tuning hyperparameters of predictive learners is also not commonly adopted in practice. In Section sec:tunesoftware of the Appendix we review several software solutions available for

hyperparameter tuning, and none included an option for blocking the seeds, confirming the lack of attention to this topic.² In some cases, an option for setting the initial random seed for the entire search is available, but this is often justified only as a means of ensuring the reproducibility of results and is not equivalent to blocking the variables. Admittedly, some documentation is sparse, but we presume that at least a mention of this practice would be made if it had been adopted.

3 A statistical model for grid search

Given a data sample $T = (X, Y)$, we consider a generic learning algorithm $f(X, \theta; R)$, with hyperparameters $\theta = (\theta_1, \theta_2, \dots)$ and a random behavior R . The CV error, denoted as $\text{CVerr}(\theta_m; R, P)$, is obtained by applying the chosen loss function to the predictions made by the learning algorithm with setting $\theta_m = (\theta_{1m}, \theta_{2m}, \dots)$ and random behavior R , to each fold of a particular CV partition P .

We aim to measure the CVerr for setting θ_m over the available data, denoted by $\text{CVerr}(\theta_m)$, for a fixed partitioning strategy, that is number of folds, restriction to instances selection, and so on. Since $\text{CVerr}(\theta_m; R, P)$ is constant when P and R are fixed, the expectation over all possible values of CVerr in the sample T is equivalent to:

$$\text{CVerr}(\theta_m) = E[\text{CVerr}(\theta_m; R, P)] = E_{R,P}[\text{CVerr}(\theta_m; R, P)|R, P], \quad (1)$$

where the subscripts under the expectation operator denote that the expectation is taken over all possible values of that variable.³ This decomposition illustrates the sources of variability in CVerr and guides experimental designs to minimize unnecessary variability.

Furthermore, also the variance of the CVerr is evaluated across CV partitions and random behaviors. Specifically, we have:

$$\begin{aligned} \text{Var}[\text{CVerr}(\theta_m; R, P)] &= E_{R,P}[\text{Var}(\text{CVerr}(\theta_m; R, P)|R, P)] \\ &+ \text{Var}_{R,P}[E(\text{CVerr}(\theta_m; R, P)|R, P)] = \text{Var}_{R,P}[\text{CVerr}(\theta_m; R, P)|R, P]. \end{aligned} \quad (2)$$

Henceforth, expectations will be considered as taken over CV partitions and random behaviors, and conditional to the training data available.

3.1 Analysis of variance

Experimental designs were first proposed in the early 19th century (Fisher, 1937), and have since been developed and refined. The general theory we present here can be found in any book on the subject, such as the classics Box et al. (2005) or Montgomery (2008).

² We included in this search most of the frameworks mentioned in Hutter et al. (2009) with the addition of some Python and R libraries and online services from big providers. The list with brief comments can be found in the Appendix.

³ In principle, we could compute $\text{CVerr}(\theta_m)$ exactly by averaging over all possible K -fold partitions and initial seeds for the learner. However, there exist $\prod_{k=1}^{K-1} \binom{n - (k-1)n_t}{n_t}$ K -fold partitions, where n is the sample size and n_t is the size of the folds (assumed constant for simplicity) and the number of possible seeds and the number of possible initial seeds is huge, as explained in footnote 4.1.1. Hence, considering all these possibilities, this computation is infeasible.

A cartesian grid built from M settings, θ_s , and blocked with respect to N_p CVseeds and N_R RBseeds is a complete balanced block design with $S \times N_p \times N_R$ runs with $\text{CVerr}(\theta_s; R, P)$ as response.

Experiments are typically analyzed by postulating an ANOVA linear model. For the blocked grid described above, the simplest such model is a first order mixed-effects model with random block effects and fixed setting effects, which can be written as

$$\text{CVerr}(\theta; R, P) = \mu + \pi + \rho + \tau + \varepsilon, \quad (3)$$

where μ is the overall mean CVerr, π and ρ are the random effects of the CV partition and random behavior, respectively, τ is the fixed effect of the settings and ε is an error term. The random terms are supposed to be mutually independent and to have regular generic distributions with

$$E(\pi) = E(\rho) = E(\varepsilon) = 0, \text{Var}(\pi) = \sigma_\pi^2, \text{Var}(\rho) = \sigma_\rho^2, \text{Var}(\varepsilon) = \sigma_\varepsilon^2. \quad (4)$$

Under this model, the expected value of $\text{CVerr}(\theta_s; R, P)$ is given by:

$$\text{CVerr}(\theta_s) = E[\text{CVerr}(\theta_s; R, P)] = \mu + \tau_s.$$

Because of the independence assumptions, the variance of $\text{CVerr}(\theta_s; R, P)$ is simply equal to the sum of the variances of the random terms, that is:

$$\text{Var}[\text{CVerr}(\theta_s; R, P)] = \text{Var}(\pi) + \text{Var}(\rho) + \text{Var}(\varepsilon).$$

In contrast to the standard assumptions made in ANOVA, where the error term ε in model (3) represents an additional noise, in our model the random components R and P are responsible for all the variability. Therefore, it is necessary to redefine the error term ε . Model (3) is derived as a first-order MacLaurin approximation of $\text{CVerr}(\theta; R, P)$. Consequently, ε can be viewed as the sum of individually nonsignificant higher-order terms. This definition suits the broad goal of learner tuning. If there is evidence indicating that higher-order terms significantly differ from zero, they can be explicitly incorporated into the model. It is important to note that while the independence assumptions are employed to simplify the analysis, they may not be entirely satisfied.

The analysis of the results of blocked experiments is based on the discrete form of model (3):

$$\text{CVerr}_{ijs}^{BCV} = \mu + \pi_i + \rho_j + \tau_s + e_{ijs}, \quad i = 1, \dots, N_p, \quad j = 1, \dots, N_R, \quad s = 1, \dots, S, \quad (5)$$

where the indices refer to the different levels of each factor chosen for the experiments. The indices i and j correspond to different CVseeds and RBseeds. When there is an interest in studying the effects of the hyperparameters the setting effects, τ_s , can be broken down into the sum of the effects of the individual hyperparameters. This breakdown can include higher-order terms such as interactions. For example,

$$\tau_s = \sum_t \gamma_t + \sum_t \sum_{s>t} \gamma_{st}.$$

Since model (5) is not identified, different restrictions can be imposed to reduce the number of parameters. We adopt the zero-sum constraints, which require that

$$\sum_i \pi_i = 0; \sum_j \rho_j = 0; \sum_s \tau_s = 0. \quad (6)$$

In a complete block balanced design, these constraints result in orthogonal contrasts and the estimates of the block effects are the differences between the block or setting means and the overall mean. That is,

$$\begin{aligned} \hat{\mu} &= \text{CVerr}_{\dots}^{BCV}, \hat{\pi}_i = (\text{CVerr}_{i\dots}^{BCV} - \text{CVerr}_{\dots}^{BCV}), \hat{\rho}_j = (\text{CVerr}_{\dots j}^{BCV} - \text{CVerr}_{\dots}^{BCV}), \\ \hat{\tau}_s &= (\text{CVerr}_{\dots s}^{BCV} - \text{CVerr}_{\dots}^{BCV}), \end{aligned}$$

where the “.” in the subscript represents the averaging over the index it replaces and the residuals $\hat{e}_{sij}$ are obtained by difference. If the effects of individual hyperparameters are included in the model, their estimates are computed in similarly. Thus, the net estimate of $\text{CVerr}(\theta)$ is the average over the random terms, R and P , that is:

$$\widehat{\text{Err}^{CV}}(\theta_s) = \text{CVerr}_{\dots s}^{BCV} = \hat{\mu} + \hat{\tau}_s,$$

where the effects of the initial seeds disappear because of the zero-sum constraints in Eq. (6).

It can be proven that

$$E[\text{CVerr}_{\dots s}^{BCV}] = \mu + \tau_s; \text{Var}[\text{CVerr}_{\dots s}^{BCV}] = \frac{\sigma_\epsilon^2}{N_P N_R}. \quad (7)$$

In RCV the CVerr^{RCV} 's are obtained by repeating N times the grid, letting the seeds vary freely instead of blocking them. Model (3) is still valid but, since the random effects are completely confounded with the error term, the empirical model becomes:

$$\text{CVerr}_{ks}^{RCV} = \mu + \tau_s + f_{ks}, \quad k = 1 \dots, N, \quad s = 1, \dots, S.$$

The error term is equal to $f_{ks} = \pi_{ks} + \rho_{ks} + e_{ks}$. The estimated CVerr^{RCV} for each setting is equal to the average over the repetitions, $\text{CVerr}_{\dots s}^{RCV}$, which can be shown to have mean and variance equal to

$$E[\text{CVerr}_{\dots s}^{RCV}] = \mu + \tau_s; \text{Var}[\text{CVerr}_{\dots s}^{RCV}] = \frac{\sigma_\pi^2 + \sigma_\rho^2 + \sigma_\epsilon^2}{N}. \quad (8)$$

Comparing the variance of the estimates obtained with CCV, Eq. (7), and with RCV, Eq. (8), it appears clear why blocking attains more precise estimates than randomization.

When a genuinely random error term is present, a design that is blocked with respect to a variable with a negligible variance is essentially equivalent to a repeated design. However, in the case of the response being the CVerr, where the sources of variability are solely attributed to R and P , blocking with respect to one of these variables can be disadvantageous if its effect is negligible. This is because the CVerr for each level of the other blocking variable are essentially identical, making the two-block design almost a redundant copy of the one-block design. Consequently, when the effects of different data partitions or algorithm's initial random seed is negligible, blocking represent an unnecessary restriction to randomization (a waste of degrees of freedom) and RCV may yield lower standard deviation.

3.2 Empirical variance breakdown

In this section, for ease of exposition, let Y represent the CVerr indexed as above. The variance of GS results can be analyzed using ANOVA, according to the models discussed previously. If the learner is trained only once for each of the S settings, the total sum of squares (SST) is given by:

$$SST = \sum_{s=1}^S (Y_s - \bar{Y})^2.$$

This sum of squares represents the variation of the CVerr due to being measured on different settings, possibly confounded with that due to P and R if these are not fixed. Therefore, it has no bearing on the variation of the CVerr for fixed settings.

In RCV, the total sum of squares of the observed CVerr for each of the N repetitions, Y_{is} , $i = 1, \dots, N$, can be decomposed as:

$$SST = \sum_{s=1}^S \sum_{i=1}^N (Y_{is} - \bar{Y}_{..})^2 = \underbrace{N \sum_{s=1}^S (\bar{Y}_{.s} - \bar{Y}_{..})^2}_{SS_{Settings}} + \underbrace{\sum_{s=1}^S \sum_{i=1}^N (Y_{is} - \bar{Y}_{.s})^2}_{SSE^{RCV}}. \quad (9)$$

Here, the residual sum of squares, SSE^{RCV} represents the sum over all settings of the squared residuals. The residuals are differences of the CVerr values when the CV partition is randomly changed around the average CVerr of the corresponding setting,

$$r_{(i)s} = Y_{is} - \bar{Y}_{.s}. \quad (10)$$

CCV is similar to RCV, with the key difference being that the CV partition is fixed for each repetition of the grid. This allows the variation due to the CV partitions to be calculated and subtracted from the residual variation as computed in RCV. When there is a single blocking variable with N_c levels, the SST decomposition is:

$$SST = \sum_{s=1}^S \sum_{i=1}^{N_c} (Y_{is} - \bar{Y}_{..})^2 = \underbrace{N_c \sum_{s=1}^S (\bar{Y}_{.s} - \bar{Y}_{..})^2}_{SS_{Settings}} + \underbrace{S \sum_{i=1}^{N_c} (\bar{Y}_{.i} - \bar{Y}_{..})^2}_{SS_{Blocks}} + \underbrace{\sum_{s=1}^S \sum_{i=1}^{N_c} (Y_{is} - \bar{Y}_{.s} - \bar{Y}_{.i} + \bar{Y}_{..})^2}_{SSE^{CCV}}. \quad (11)$$

It is trivial to prove that:

$$\begin{aligned} SS_{Blocks} + SSE^{CCV} &= S \sum_{i=1}^{N_c} (\bar{Y}_{.i} - \bar{Y}_{..})^2 + \sum_{s=1}^S \sum_{i=1}^{N_c} (Y_{is} - \bar{Y}_{.s} - \bar{Y}_{.i} + \bar{Y}_{..})^2 \\ &= \sum_{s=1}^S \sum_{i=1}^{N_c} (Y_{is} - \bar{Y}_{.s})^2 = SSE^{RCV}. \end{aligned}$$

Hence, by comparing this expression with the expression of SSE for RCV in Eq. (9), it becomes evident how blocking reduces the variance of the residuals. The decomposition for two or more blocking variables is analogous and can be found in any standard reference on the topic.

When seeds are blocked, the residuals differ from those defined in Eq. 9 for RCV. In CCV, each observation is adjusted for the effect of the CV partition under which it was obtained, $(\bar{Y}_{i.} - \bar{Y}_{..})$, providing a measure of how well the data were predicted under that particular partition relative to the average prediction error. The difference with the residuals computed in RCV becomes more evident when writing the CCV residuals in Eq. 11 as:

$$r_{(i)s} = [Y_{is} - (\bar{Y}_{i.} - \bar{Y}_{..})] - \bar{Y}_{..s}. \quad (12)$$

When the grid is also blocked with the seeds for random behavior, the residuals are equal to:

$$r_{(ij)s} = [Y_{ijs} - (\bar{Y}_{i..} - \bar{Y}_{...}) - (\bar{Y}_{.j.} - \bar{Y}_{...})] - \bar{Y}_{...s}, \quad (13)$$

so they are adjusted also for the random behaviour effects as

$$r_{(ij)s} = \text{CVerr}_{ijs}^{BCV} - \hat{\pi}_i - \hat{\rho}_j = \text{CVerr}_{..s}^{BCV} + \hat{e}_{ijs},$$

where $\hat{\pi}_i = (\bar{Y}_{i..} - \bar{Y}_{...})$ and $\hat{\rho}_j = (\bar{Y}_{.j.} - \bar{Y}_{...})$ are the estimates of the corresponding factor effects.

3.2.1 Variance in terms of squared differences

Lemma 1 Given a set of N continuous observations $Y_1, Y_2, \dots, Y_N$, define $\bar{Y} = \sum_{i=1}^N Y_i / N$ as the sample mean. The variance of these observations can be expressed using the sum of their squared differences as:

$$\sum_{i=1}^N (Y_i - \bar{Y})^2 = \frac{1}{2N} \sum_{i=1}^N \sum_{j=1}^N (Y_i - Y_j)^2 \quad (14)$$

$$\begin{aligned} \sum_{i=1}^N \sum_{j=1}^N (Y_i - Y_j)^2 &= \sum_{i=1}^N \sum_{j=1}^N Y_i^2 - 2 \sum_{i=1}^N \sum_{j=1}^N Y_i Y_j + \sum_{i=1}^N \sum_{j=1}^N Y_j^2 \\ &= 2N \sum_{i=1}^N Y_i^2 - 2N^2 \bar{Y}^2 = 2N \sum_{i=1}^N (Y_i - \bar{Y})^2, \end{aligned}$$

Proof

because $\sum_{i=1}^N Y_i = N\bar{Y}$ and $\sum_{i=1}^N \sum_{j=1}^N Y_i Y_j = \sum_{i=1}^N Y_i \sum_{j=1}^N Y_j = N^2 \bar{Y}^2$, and $\sum_{i=1}^N Y_i^2 - N\bar{Y}^2 = \sum_{i=1}^N (Y_i - \bar{Y})^2$. $\square$

Corollary 1 Eq. (14) can be simplified by removing duplicated terms and differences involving equal elements as:

$$\sum_{i=1}^N (Y_i - \bar{Y})^2 = \frac{1}{N} \sum_{i=1}^{N-1} \sum_{j=i+1}^N (Y_i - Y_j)^2$$

$\square$

Corollary 2 *Lemma 1 can be extended to the total variance of observations Y_{is} indexed by two variables, each having N and S levels, respectively, by iterative application of Eq. (14). Let the partial and total means be defined as*

$$\bar{Y}_{i.} = \frac{1}{S} \sum_{s=1}^S Y_{is}; \quad \bar{Y}_{.s} = \frac{1}{N} \sum_{i=1}^N Y_{is}; \quad \bar{Y}_{..} = \frac{1}{N} \sum_{s=1}^S \sum_{i=1}^N Y_{is},$$

then, the total variance becomes

$$SST = \sum_{s=1}^S \sum_{i=1}^N (Y_{is} - \bar{Y}_{..})^2 = \frac{1}{2SN} \sum_{s=1}^S \sum_{t=1}^S \sum_{i=1}^N \sum_{j=1}^N (Y_{is} - Y_{jt})^2.$$

Furthermore, by proceeding as in Corollary 1, the formula can be simplified by removing duplicated terms and differences of identical elements as

$$\sum_{s=1}^S \sum_{i=1}^N (Y_{is} - \bar{Y}_{..})^2 = \frac{1}{SN} \sum_{s=1}^{S-1} \sum_{t=s+1}^S \sum_{i=1}^{N-1} \sum_{j=i+1}^N (Y_{is} - Y_{jt})^2. \quad (15)$$

The extension can be further generalized to variables with three or more indices. □

In light of Corollary 2, the variance components of the RCV and CCV designs can be expressed as follows:

$$SSE_{\text{settings}} = N \sum_{s=1}^S (\bar{Y}_{.s} - \bar{Y}_{..})^2 = \frac{N}{2S} \sum_{s=1}^S \sum_{t=1}^S (\bar{Y}_{.s} - \bar{Y}_{.t})^2 \quad (16)$$

$$SS_{\text{Blocks}} = S \sum_{i=1}^{N_c} (\bar{Y}_{i.} - \bar{Y}_{..})^2 = \frac{S}{2N} \sum_{i=1}^{N_c} \sum_{j=1}^{N_c} (\bar{Y}_{i.} - \bar{Y}_{j.})^2 \quad (17)$$

$$SSE^{\text{RCV}} = \sum_{s=1}^S \sum_{i=1}^N (Y_{is} - \bar{Y}_{.s})^2 = \frac{1}{2N} \sum_{s=1}^S \sum_{i=1}^N \sum_{j=1}^N (Y_{is} - Y_{js})^2 \quad (18)$$

$$SSE^{\text{CCV}} = \sum_{s=1}^S \sum_{i=1}^{N_c} (Y_{is} - \bar{Y}_{i.} - \bar{Y}_{.s} + \bar{Y}_{..})^2 = \frac{1}{2N} \sum_{s=1}^S \sum_{i=1}^{N_c} \sum_{j=1}^{N_c} [(Y_{is} - \bar{Y}_{i.}) - (Y_{js} - \bar{Y}_{j.})]^2, \quad (19)$$

where the last equality can be derived by substituting $W_{is} = (Y_{is} - \bar{Y}_{i.})$, so that $\bar{W}_{i.} = \bar{Y}_{.s} - \bar{Y}_{..}$, and then applying Eq. (14).

The importance of blocking the repetitions of the grid can be understood by comparing Eq. (18) with Eq. (19). In CCV designs, the SSE is computed as the sum of squared differences for each setting across all CVerrs. In contrast, in RCV designs, this sum is calculated for the differences between observations adjusted to remove the block effects.

Table 1 Components of an ANOVA table for a grid-search with S settings, RCV with N repetitions, and CCV blocked by N_c CV seeds and N_r RB seeds

Source	RCV	CCV df	SS	MS	p -value
CV seed	—	$N_c - 1$		SS/df	
RB seed	—	$N_r - 1$		SS/df	
Settings	$S - 1$	$S - 1$		SS/df	
Residuals	$N(S - 1)$	$SN_cN_r - S - N_c - N_r + 2$		SS/df	

3.3 The analysis of variance

The Mean Squares (MSs) are the estimates of the variance due to the corresponding factor, given in Eq. 4, and computed by dividing the sum of squares by the corresponding degrees of freedom. The Mean Square error, MSE_{Res} , is an estimate of the variance of a single computed CVerr, Y_{ijs} , σ_ε^2 . A typical ANOVA table is shown in Table 1:

The p -values in the ANOVA table (Table 1) represent the empirical significance levels for testing the null hypothesis that the effect of the corresponding factor is zero. The formulae for the sum of squares is omitted in Table 1 and can be found in Equations (9) and (11). When the number of repetitions N is the same as the number of blocks, N_cN_r in the tables above, the degrees of freedom of the residual sum of squares, SSE , of CCV are fewer than in RCV. This reflects the restriction to randomization due to blocking.

The standard deviation of the estimated mean CVerr for each setting s , $\bar{Y}_{.s}$, is traditionally called its *standard error* and will be denoted generically as CVstderr. It is computed as the square root of the MSE divided by the number of times the grid was replicated:

$$\text{std.err}(\bar{Y}_{.s}) = \sqrt{\frac{MSE}{N}} \text{ for RCV, and } \text{std.err}(\bar{Y}_{.s}) = \sqrt{\frac{MSE}{N_cN_r}} \text{ for CCV.} \quad (20)$$

3.4 Permutation tests

In classical ANOVA, the estimated effects are tested by comparing the ratio of their MSE and the residual MSE to an F-distribution. However, in our setup, we cannot rely on F-tests because all of the variance of the CVerr is accounted for by the nuisance variables, and we do not want to make assumptions about the distribution of the errors.

To address this issue, permutation tests can be used as a nonparametric alternative. These tests compute the p -values as the proportion of times a statistic would be larger than that observed after permuting the data. The results rely on the weak exchangeability of the observations under the null hypotheses that one or more effects are zero, and they ignore the variance of the error. Therefore, they can be performed on dependent observations, provided that the exchangeability assumption is valid. An important difference between permutation and traditional significance tests is that the former do not compare the size of

Table 2 Datasets used in the examples

Dataset	Dimensions		Task	
	Cases	Features	Classification	Regression
Sonar	208	60	Binary	
Penguins	342	5	3 Classes	
Boston	506	13		✓
Insurance	1338	6		✓
Wine	4898	11	7 Classes	✓
Adults	30162	13	Binary	

the variance of a factor's effect to that of the residual variance. A useful reference for permutation tests is provided by Good (2013).

Permutation tests for blocked experimental designs can be challenging because of the restrictions on exchangeability introduced by the blocking factors. Various data transformations have been proposed to enable exchangeability for these tests (see Frossard & Renaud, 2021, for a concise overview and references). On our balanced complete block designs with orthogonal contrasts, all of these data transformations reduce to the seminal method of Kennedy (1995). This method effectively boils down to permuting the residuals described by equations (10), (12) and (13).

4 Real data examples

In this section, we present examples of hyperparameter tuning using Grid Search, which involves exploring a Cartesian grid of all possible combinations of the selected hyperparameter values, as specified later. We compare the standard error of the estimated mean CVerr obtained using CCV and RCV for hyperparameter tuning of three learners: k-Nearest Neighbors (k-NN), Random Forest (RF), and Boosted Trees (Light Gradient Boosting, LGB). These comparisons are performed on six small to medium-sized real-world datasets, summarized in Table 2. More detailed descriptions of the datasets are provided in Sect. 7.1 of the Appendix.

As predicted by our theoretical analysis, CCV consistently outperformed RCV across computations, though the magnitude of improvement varied depending on the dataset and learner. It is important to emphasize that these examples do not serve as proof of CCV's superiority over RCV. The numerical results presented are based on a limited set of random seeds, as detailed in Sect. 4.1.1, and do not represent genuine attempts to optimize learners for each dataset. Our primary objective is to compare the CVstderr achieved under different CV designs.

While adaptive tuning strategies typically yield better results than grid search, we opted for grid search due to its simplicity, as highlighted in the introduction, even using the same parameter grid across datasets whenever feasible. Consequently, these results illustrate the differences among CV designs when applied to various learners and datasets of small to medium size, rather than optimal performance for each learner.

Table 3 Initial seeds used for different variance reduction designs on a grid with S settings, using a CV Partition blocks, b and c random behaviour blocks, or N repetitions

Design	Random source		
	CVSeeds	RBseeds 1	RBseeds 2
CCVaxbxc	1:a	1:b	1:c
CCVax0x0	1:a	1:(aS)	1:(aS)
RCV N	1:NS	1:NS	1:NS

4.1 Experimental setup

Of the three learners considered, k-NN is a deterministic algorithm, meaning it does not involve random behavior. The experiments for this learner were conducted using the `knn` function contained in the R package `Rfast`. In contrast, RF and LGB employ two sources of randomization: subsampling the data for tree construction (bagging) and selecting features at each node. The `ranger` function from the R package `ranger` (Wright & Ziegler, 2017), used for RF computations, permits the use of only a single random seed to control both subsampling and feature selection. Conversely, the `lightgbm` function in the R package `lightgbm` (Ke et al., 2017), employed for LGB predictions, allows separate seeds to be set for these two random choices.

For k-NN, only the effect of blocking CV partitions can be evaluated due to its deterministic nature. For RF and LGB, we explored two distinct strategies:

1. Comparing RCV with CCV when only the CV partition is blocked (CCVax0x0). In this setup, the initial random seed for RF and LGB varied for each trial but remained consistent across designs, isolating the effect of blocking CV partitions.
2. Comparing RCV with CCV when both the CV partition and random behavior are blocked. This was implemented for RF and LGB using CCVaxbx0 and CCVaxbxc, respectively. In this setup, the effects of blocking random behavior were also assessed.

The computations were performed using the R language (R Core Team, 2020) with our own implementation of CCV and RCV. In our implementation, blocking the initial seeds of the algorithms extends to the computation of the error in each fold, meaning the `CVerr` is computed on each fold using the same initial seed. The permutation tests were conducted using the `lmPerm` R package (Wheeler & Torchiano, 2016). All computations were run on a quad-core 11-th generation Intel® Core(TM) 2.8GHz CPU with 16GB of RAM.

4.1.1 Random seeds selection

For reproducibility, the initial seeds were set as linear sequences starting from one. In CCV, the CVseeds and, in multiple blocking designs, RBseeds were blocked. In RCV, the initial seeds were different for each run. Table 3 shows the seeds used for applying CCV blocking with two random behavior seeds. The seed selection for cases with blocking only one or no random behavior seeds can be obtained by omitting the unnecessary seeds from the table.

Note that when the number of CV blocks (a) in CCV ax0x0 equals the number of repetitions (N) in RCV, the initial seeds controlling random behavior remain identical across

Table 4 k-NN: CVstderrs obtained with the CCV12 and RCV12 designs. The first columns shows the Dataset, the next two columns show the CVstderr for the two designs, the fourth column their percent ratio and the fifth the p -value of the CVseeds effects

Dataset	CVstderr		%Ratio CVstderr CCV/RCV (%)	p -value
	CCV	RCV		
Sonar	0.003487	0.004759	73	0.000
Penguins	0.002138	0.004333	49	0.000
Boston	0.018287	0.031725	58	0.000
Insurance	0.459608	0.732427	63	0.000
Wine reg	0.001309	0.002215	59	0.000
Wine class	0.000585	0.000844	69	0.000
Adults	0.000267	0.000341	78	0.000

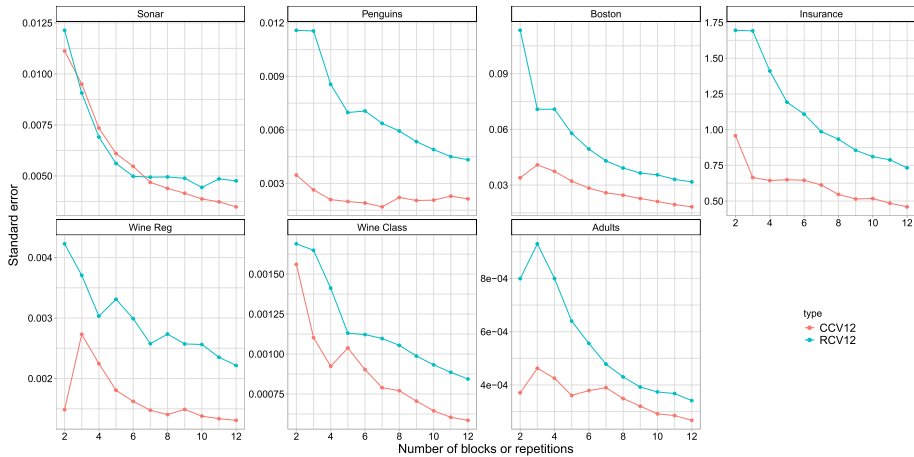


Fig. 1 k-NN: standard error of mean CVerr as the number of blocks or repetitions increases

all runs. This consistency holds even when subsets with fewer repetitions or blocks are analyzed.

4.2 Results

We now give a description of the results obtained separately for each learner considered. As mentioned, we only illustrate the key results. More exhaustive tables can be found in Sect. 8 of the Appendix.

4.2.1 k-Nearest neighbors

In the simple deterministic k-NN algorithm (fix, 1951) there are only two parameters that can be tuned: distance measure, typically Manhattan or Euclidean, and number of neighbouring values considered. So we ran two designs: Controlled CV with 12 blocks CCV12,

Table 5 Values of the hyperparameters used to build the grid for tuning the random forests for each dataset

Dataset	Parameter values			
	mtry	min.node.size	replace	sample.fraction
Sonar	5 10 20	3 5 10 15	TRUE FALSE	0.5 0.7 0.9 1.0
Penguins	2 3 5	1 3 5 10	TRUE FALSE	0.5 0.7 0.9 1.0
Boston	3 7 11	3 5 10 15	TRUE FALSE	0.5 0.7 0.9 1.0
Insurance	2 4 6	3 5 10 15	TRUE FALSE	0.5 0.7 0.9 1.0
Wine	3 6 9	3 5 10 20	TRUE FALSE	0.5 0.7 0.9 1.0
Adults	3 6 12	1 3 5 10	TRUE FALSE	0.5 0.7 0.9 1.0

blocking the 12 repetitions with 12 different CVseeds, and Repeated CV with 12 repetitions RCV12, in which the grid was repeated 12 times, at each run using a different CVseed.

Table 4 shows the CVstderr obtained with the CCV12 and RCV12 designs for all datasets. The table highlights the consistent reduction in CVstderr achieved by CCV12 compared to RCV12 across all datasets, with notable improvements such as a 51% reduction for Penguins (0.002138 vs. 0.004333) and a 42% reduction for Boston (0.018287 vs. 0.031725). The p -values for CVseeds are uniformly 0.000, indicating a significant effect of CV partitioning on variability. These results demonstrate the superior precision of CCV12 in controlling variability.

The CVstderr obtained with increasing number of blocks or repetitions are shown in Fig. 1. The chart illustrates the standard errors of the mean CVerr for the seven examples under the RCV12 and CCV12. For RCV12, the standard error generally decreases as the number of repetitions increases, although the rate of decline varies across datasets. For instance, the Penguins dataset exhibits a sharper decline, while datasets like Wine Class show a more gradual reduction. However, occasional increases in standard error are observed for some datasets, such as in RCV12 for the Sonar dataset, where variability slightly increases at intermediate repetition levels before stabilizing.

In contrast, CCV12 consistently achieves lower standard errors than RCV12 for the same number of blocks or repetitions, particularly for examples like Penguins and Wine Reg, where the standard error starts low and decreases rapidly. In most cases, CCV12 shows a steeper decline in standard error compared to RCV12, reaching stability with fewer blocks. Interestingly, for CCV12, the Sonar dataset displays a minor increase in standard error at higher block counts, suggesting some residual variability. Overall, these results underscore that CCV12 is more effective in controlling variability and is a preferable design for obtaining precise estimates of the mean CVerr across diverse datasets.

4.2.2 Random forest

The RF algorithm implemented in the *ranger* function includes several “early stopping” parameters, some of which are interchangeable. The four hyperparameter considered for tuning with the ranges chosen to build the grids are shown in Table 5. The values for *mtry* (the number of features tried at each node) and *min.node.size* (the minimum number of cases allowed in a node) were adjusted based on each dataset’s feature count to maintain similar ratios across datasets. The *replace* parameter determines whether subsampling for bagging is performed with or without replacement, while *sample.fraction* specifies the fraction of cases used for subsampling. Settings with *replace* = *FALSE* and *sample.fraction* = 1

Table 6 Values of the fixed hyperparameters used for tuning the random forests for each dataset

Parameter	Value
num.trees	500
min.bucket	1
max.depth	Unlimited
case.weights	None, equal weight for each case
class.weights	None, equal weight for each class
splitrule	“Gini” for classification, “variance” for regression
split.select.weights	None, equal weight for each variable
always.split.variables	None
respect.unordered.factors	Order, unordered factors are ordered according to methods explained in the documentation ranger package documentation

were excluded, resulting in a grid with 84 settings for each dataset. The hyperparameters that were kept constant are listed in Table 6. Their names are self-explanatory.

We ran three designs with 32 repetitions or blocks: CCV8x4, blocking eight CVseeds and four RBseeds; CCV32x0 blocking only 32 CVseeds; and RCV32 with 32 repetitions of the grid in which each the CVerr was computed using different random seeds. The seed assignment details are provided in Table 3. Table 7 shows the standard errors for different designs, their ratios with that of RCV and the p -values of the different random seeds blocked in that design. Both blocked designs attained lower CVstderr than RCV. In some cases with ratios close to a third (CCV8x4 for the regression on the Wine dataset) or a quarter (CCV32x0 on the Boston dataset). CCV8x4 attains slightly lower CVstderr than CCV32x0 for all datasets, with the exception of the Sonar datasets. The effect of the CVseeds are significantly different from zero (negligible p -value) for both blocked designs, while the effect of the RBseeds, blocked only in CCV8x4, have p -value lower than 0.1 only for the Insurance dataset and the regression on the Wine dataset, indicating that this seed does not substantially increase the variance in most cases.

Figure 2 displays the CVstderr values across the seven examples for the three designs considered: RCV32 (blue square marks), CCV32x0 (green triangular marks), and CCV8x4 (orange circular marks). The x-axis represents the number of blocks or repetitions, while the y-axis shows the standard error, CVstderr. Across all datasets, as the number of runs increases, the values for the two CCV designs (CCV32x0 and CCV8x4) converge, highlighting their similar behavior at higher run counts. RCV32, however, consistently exhibits higher standard errors, with slower decreases as the number of runs increases, indicating less effective control of variability.

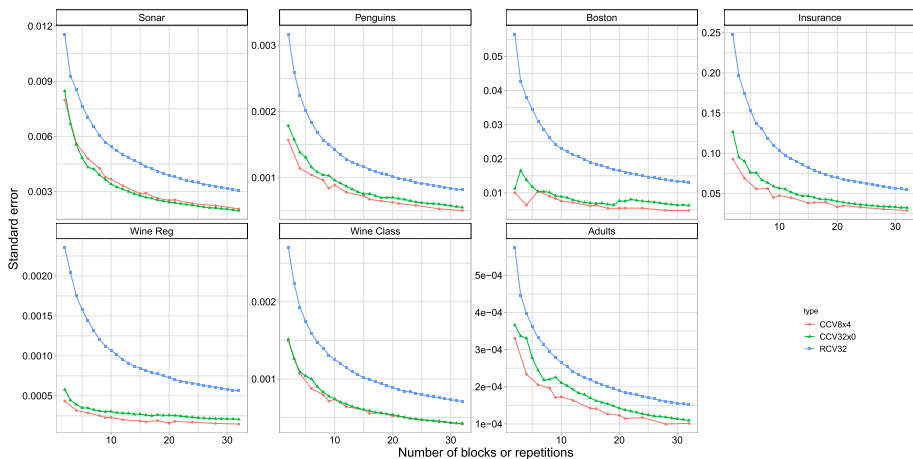
CCV8x4 consistently achieves the lowest standard errors across most datasets, particularly at lower run counts, reflecting its robust performance when controlling both CV partitioning and random behavior. CCV32x0 also outperforms RCV32 but starts with slightly higher standard errors than CCV8x4. Despite these differences, the two CCV designs converge as the number of runs increases. RCV32 shows less stability, with higher standard errors and slower reductions across all datasets, making it the least effective design.

For Wine Reg, CCV8x4 starts with the lowest standard error, declines steeply, and converges with CCV32x0, while RCV32 remains higher. In Wine Class, CCV8x4 initially declines faster than CCV32x0, though both converge at higher runs, while RCV32 remains

Table 7 Summary statistics for the tuning of Random Forests for all datasets using the blocked designs CCV8x4 and CCV32x0, and RCV32

Dataset	CVstderr			%Ratios CVstderr		<i>p</i> -values		
	CCV8x4	CCV32x0	RCV32	CCV8x4/ RCV32 (%)	CCV32x0/ RCV32 (%)	CVSeeds CCV8x4	CVSeeds CCV32x0	RBSeeds CCV8x4
Sonar	0.002061	0.001970	0.003056	67.4	64.5	0	0	0.119
Penguins	0.000503	0.000552	0.000818	61.5	67.5	0	0	0.355
Boston	0.004809	0.006315	0.012984	37.0	48.6	0	0	0.412
Insurance	0.028747	0.032302	0.054687	52.6	59.1	0	0	0.002
Wine reg	0.000146	0.000205	0.000566	25.8	36.3	0	0	0.031
Wine class	0.000416	0.000425	0.000709	58.6	59.9	0	0	0.279
Adults	0.000102	0.000109	0.000152	67.0	72.2	0	0	0.993

The first three columns show the average CVstderrs attained, the next two columns the percentage ratios of the CVstderrs attained with CCV and that attained with RCV. The last three columns show the *p*-values of the CVseeds and RBseeds for the blocked designs

**Fig. 2** RF: standard error of mean CVerr as the number of blocks or repetitions increases

relatively high. For Adults, CCV designs decline rapidly and converge, with CCV8x4 slightly outperforming CCV32x0, while RCV32 lags behind. In Sonar, CCV8x4 achieves the lowest standard error, closely followed by CCV32x0, whereas RCV32 remains highest. Penguins show CCV8x4 consistently lowest, converging with CCV32x0, while RCV32 improves slowly. In Boston, CCV8x4 maintains slightly lower values than CCV32x0, both outperforming RCV32. For Insurance, CCV8x4 starts low and converges with CCV32x0, while RCV32 shows slower reductions.

Table 8 Parameters values for LGB

Tuned		Fixed	
Parameter	Values	Parameter	Value
Bagging_fraction	0.5, 0.75, 1	Feature_fraction	0.75
Llearning_rate	0.05, 0.1, 0.3, 0.5	Min_data	20
Num_iterations	50, 120	Bagging_freq	1
		Max_depth	10
		Num_leaves	15

4.2.3 Gradient boosting machine

The `lightgbm` function from the R package `lightgbm` (Ke et al., 2017), used in our computations, allows for two initial random seeds: one for subsampling used to build each tree (Sseed) and another for sampling the features tried at each node (Fseed).

To evaluate the impact of blocking, we experimented with three designs. First, we blocked the grid using eight CVseeds, four Sseeds, and four Fseeds, resulting in a total of 128 blocks (denoted as CCV8x4x4). Second, we blocked the grid using only 128 CVseeds (CCV128x0), without blocking the random behavior seeds. Finally, for comparison, we performed 128 replications where different seeds were used for each run without blocking (RCV128). The seed assignment details are provided in Table 3.

For each example, we tuned three parameters, resulting in 24 settings. These parameter values, along with those fixed during the experiments, are shown in Table 8.

Table 9 provides a summary of the tuning results for LGB. The CVstderr values for CCV8x4x4 and CCV128x0 are consistently lower than those for RCV128 across all datasets but for the Wine Class example. Notable reductions are observed in the Penguins dataset, where CCV8x4x4 achieves 49% of the RCV128 standard error, and in the Insurance dataset, where it achieves 77%.

The performance of CCV128x0 is generally competitive with CCV8x4x4, with percentage ratios often close to 100%. This indicates that CCV128x0 is effective at reducing standard errors without the need for additional random blocking. For example, the ratios for the Adults dataset are 93% for CCV8x4x4 and 94% for CCV128x0, showing minimal difference between the two designs.

The proportional gains in CVstderr of blocked designs over RCV are not as substantial as those seen in the RF case. This is largely due to the unrealistically large number of runs considered, which naturally diminishes the observable advantages of blocking over repetition.

The p -values for CVseeds are consistently negligible (0 or near 0) for both CCV designs, demonstrating that the variability due to CVseeds is significant across datasets. However, the effects of the additional random blocking seeds (Sseed and Fseed) vary. In the Penguins dataset, the p -value for Fseed is 0.014, suggesting some influence, while Sseed has a minimal effect with a p -value of 0.135. For the Insurance dataset, Sseed shows marginal significance (p -value = 0.060), but Fseed has a strong effect (p -value = 0). For most other datasets, the effects of Sseed and Fseed are negligible, with p -values close to 1.0.

Overall, the results demonstrate that while blocked designs (CCV8x4x4 and CCV128x0) effectively reduce variance compared to RCV128, the advantages become less pronounced with a large number of runs, as expected. This highlights the diminishing returns of

Table 9 Summary statistics for the tuning of Light Gradient Boosting Machines for all datasets using the blocked designs CCV8x4x4 and CCV128x0, and RCV128. The first three columns show the average standard errors attained, the next two columns the percentage ratios of the standard errors attained with CCV and that attained with RCV. The last three columns show the *p*-values of the CV-seeds and RB-seeds for the blocked designs

Dataset	CV stderr	%Ratios CV stderr					<i>p</i> -values		
		CCV8x4x4/		CCV128x0/		RCV128 (%)	CV seed		
		RCV128 (%)	RCV128 (%)	CCV128x0	CCV8x4x4		CCV 8x4x4	CCV 128x0	Fseed
Sonar	0.00125	0.00135	0.00166	76	81	0	0	0	0
Penguins	0.00028	0.00054	0.00057	49	95	0	0	0	0.014
Boston	0.00829	0.00751	0.01091	76	69	0	0	0	0
Insurance	0.20341	0.19486	0.26415	77	74	0	0	0	0
Wine reg	0.00039	0.00041	0.00048	81	86	0	0	0	0.003
Wine class	0.00223	0.00220	0.00223	100	99	0	0	0.600	0
Adults	0.00007	0.00007	0.00008	93	94	0	0	0	0

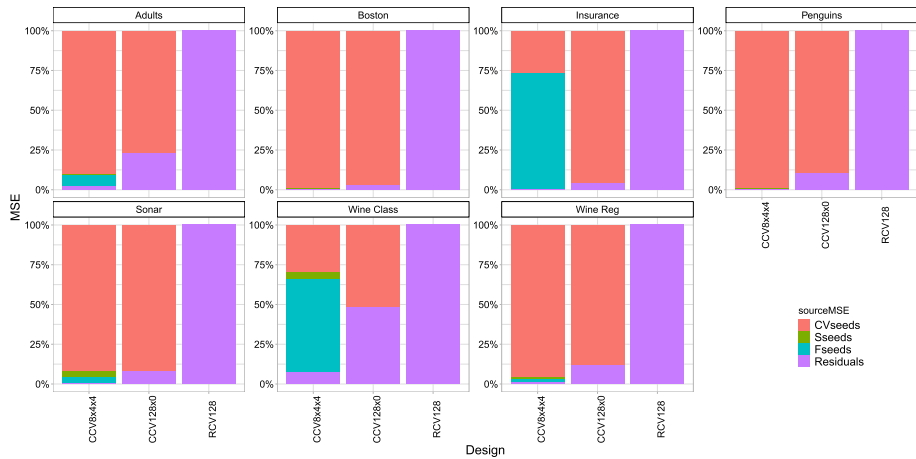


Fig. 3 LGB: proportion of variance explained by the random terms for all designs and datasets

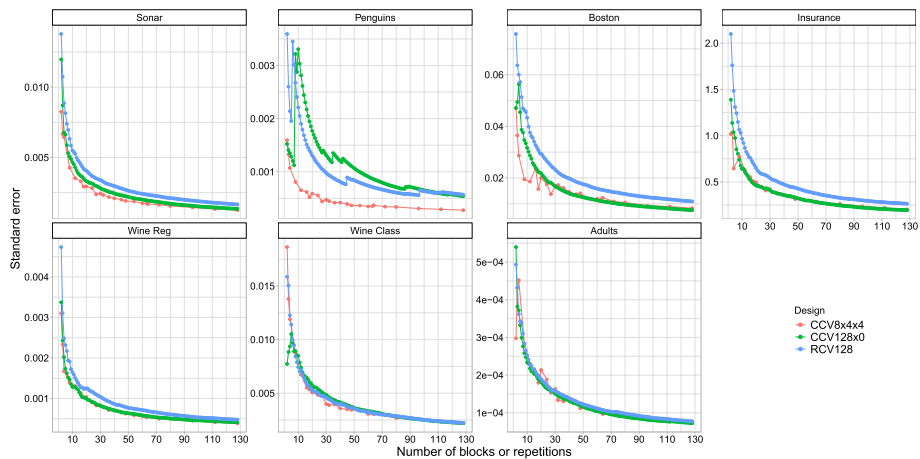


Fig. 4 LGB: standard error of mean CVerr as the number of blocks or repetitions increases

blocking under these conditions but underscores the robustness of CCV in achieving lower variability across different datasets.

The proportion of variance (MS) attributed to different random terms—CV partitions (red), subsampling (blue), feature selection (blue), and residuals (purple)—is illustrated in Fig. 3. In the RCV design, the MSE proportion is necessarily equal to one, as there are no blocking variables to partition the variance. For CCV128x0, only the CVseeds are explicitly measured, while contributions from Sseeds and Fseeds are confounded with the residuals.

CVseeds are the dominant source of variance across most datasets, especially in Boston, Penguins, Sonar, and Wine Reg. This highlights the significant impact of CV partitioning on variability. In the CCV8x4x4 design, Fseeds surpass CVseeds in variance contribution

for the Insurance and Wine Class datasets, whereas Sseeds play a smaller but still notable role in Sonar and Wine Class, indicating some additional unexplained variability.

For CCV128x0, the residual contributions are minimal in most datasets but are substantially larger in Wine Class and moderately higher in Adults. This increase is likely due to the confounding of random behavior with residuals, which inflates the apparent contribution of residuals. Despite this, CVseeds consistently emerge as the primary source of variance across all datasets. The CCV8x4x4 design, with its finer decomposition, underscores the crucial role of CVseeds in determining variability and provides a more granular understanding of variance contributions compared to CCV128x0.

Fig. 4 displays the CVstderr values computed with LGB for the seven examples, plotted against the number of blocks or repetitions. Three designs are represented: CCV8x4x4 (red circles), CCV128x0 (green triangles), and RCV128 (blue squares). The x-axis indicates the number of blocks or repetitions, while the y-axis shows the standard error for each design and dataset.

In most examples, CVstderr decreases smoothly as the number of blocks or repetitions increases, with a slowing rate of decrease. Both blocked designs consistently achieve lower CVstderr than RCV for the same number of blocks or repetitions, with a varying degree of improvement, for the larger datasets, Wine and Adults, the CVstderrs are very close. With the exception of the Penguin dataset, CCV8x4x4 and CCV128x0 yield similar CVstderr values but the former shows a rougher decrease, possibly due to having removed some values obtain the same number of blocks.

For CCV8x4x4, the standard error generally starts low (except for Wine Class) and declines rapidly across all examples, reflecting robust control of variability. This design consistently achieves the lowest values, particularly at lower block counts, and demonstrates its effectiveness in managing variability by addressing multiple sources of randomness. Its steep initial decline suggests it achieves stability more quickly.

CCV128x0 shows a similar downward trend but starts slightly higher than CCV8x4x4 in most examples and converges toward comparable values as the number of blocks increases. While it outperforms RCV128, it does not match CCV8x4x4's early performance. RCV128, in contrast, reduces standard error more slowly, maintains higher values compared to both CCV designs, and exhibits greater variability across datasets. Some datasets even show minor fluctuations in standard error for RCV128, underscoring its inconsistency across runs. Overall, CCV designs, particularly CCV8x4x4, are more reliable and efficient in minimizing the standard error compared to RCV128.

For Wine Reg and Insurance, CCV8x4x4 shows a significant early advantage, starting with much lower standard errors and converging with CCV128x0 as the number of blocks increases. In Wine Class and Sonar, CCV8x4x4 and CCV128x0 exhibit similar trends, but RCV128 maintains higher standard errors throughout. Penguins demonstrates a steeper decline for CCV8x4x4, with CCV128x0 following closely, while RCV128 improves more slowly. For Adults, all designs show a decline, but CCV8x4x4 clearly outperforms the others, particularly at lower runs. In Boston, the performance gap between CCV8x4x4 and CCV128x0 narrows significantly as blocks increase, though both remain far better than RCV128.

This analysis confirms CCV8x4x4's superiority in controlling variability, particularly at lower run counts, while CCV128x0 provides a strong alternative. RCV128, however, shows limited effectiveness, with consistently higher standard errors and slower improvements.

Figure 5 is generated using the same data shown in Fig. 4 but zooms in on the results for blocks or repetitions of 96 or more. Hence, it examines results for an extremely large number of blocks or repetitions, far exceeding what is typically used in practice.

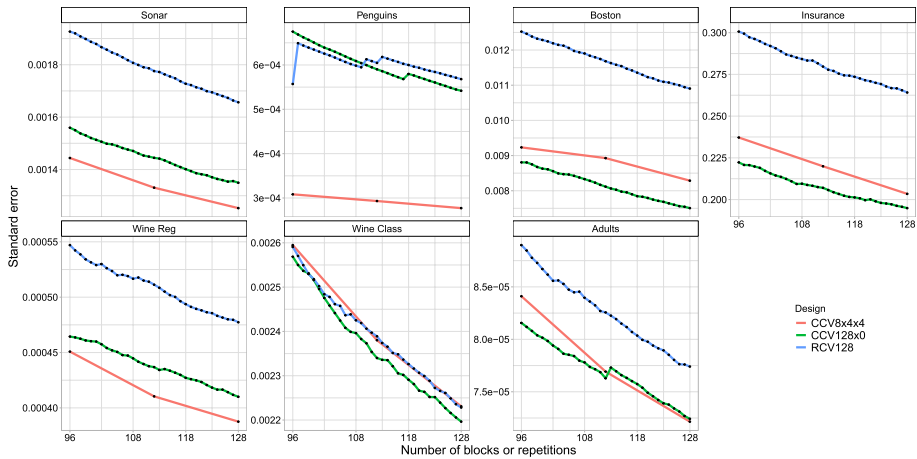


Fig. 5 LGB: CVstderr when an increasing number of blocks or repetitions are included. Last 30 values

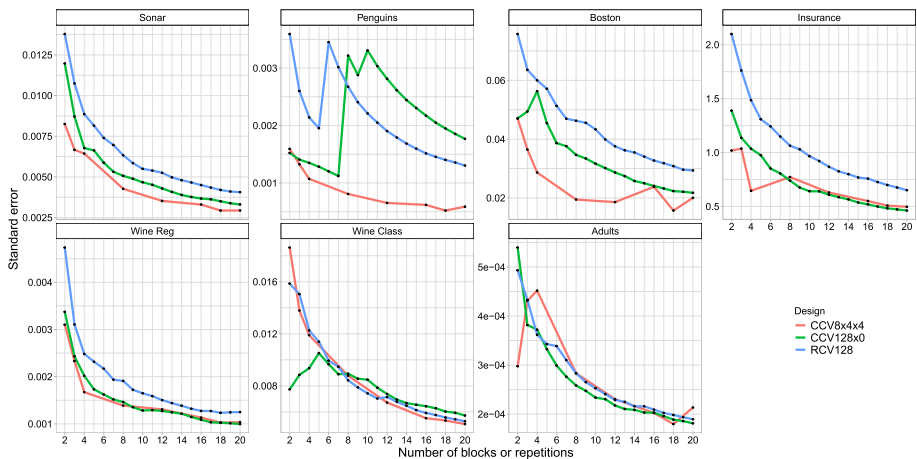


Fig. 6 LGB: standard error of the mean CVerr when an increasing number of blocks or repetitions are included. First 20 values

For CCV8x4x4, the standard errors remain the lowest across all datasets, with minimal variation, even at this high number of runs. CCV128x0 converges closely with CCV8x4x4, showing only slightly higher standard errors, reflecting robust performance at higher block counts. In contrast, RCV128 consistently has the highest standard errors, with only modest declines as repetitions increase, indicating its weaker ability to control variability compared to the CCV designs. General Observations:

While the differences in standard error between CCV8x4x4 and CCV128x0 become negligible at this extreme range of runs, CCV8x4x4 maintains a slight advantage. The results emphasize the limitations of RCV128, which continues to exhibit significantly higher variability. However, the number of blocks or repetitions considered here is exceptionally large and unlikely to be used in practical scenarios, making these observations more relevant for theoretical comparisons than real-world applications.

In most applications tuning is done using a limited number of repetitions, so it is interesting to observe the results obtained with a low number of blocks or repetitions. Figure 6, derived from the same data as previous plots, using a more realistic number of blocks or repetitions, ranging from 2 to 20.

For CCV8x4x4, the standard errors start low and decrease rapidly across all datasets, highlighting its ability to control variability effectively, even with a small number of runs. CCV128x0 also demonstrates a noticeable decline in standard errors but starts slightly higher than CCV8x4x4, with the difference between the two designs more pronounced in this range. RCV128, in contrast, begins with significantly higher standard errors and shows slower improvement as repetitions increase, indicating limited ability to stabilize variability in this realistic setting.

The smaller number of runs emphasizes the clear advantage of CCV designs, particularly CCV8x4x4, which consistently achieves the lowest standard errors across datasets with a rapid decline as the number of blocks increases. While CCV128x0 performs well, its standard errors are consistently higher than CCV8x4x4, especially in early runs, underscoring the benefit of the additional blocking in CCV8x4x4. RCV128 lags behind both CCV designs, with slower reductions and higher variability. This chart highlights the practical superiority of CCV8x4x4 for managing variability with fewer runs, making it the most reliable choice in realistic scenarios.

4.2.4 Investigation of unusual values due to the choice of random seeds

In Fig. 6 can be observed two sharp increases in the CVstderr for the Penguin dataset: one at six runs for RCV128 and another at height runs for CCV128x0. We now investigate their cause.

Both increases in the CVstderr in the sixth repetition of RCV and the eighth block of CCV128x0 are caused by unusually large CVerr values for setting number 22, as shown in Table 10.

Figure 7 presents the distribution of individual CVerr values across the 24 settings used in all designs with 128 blocks or repetitions. Box plots for settings 10 and 22, highlighted in red, provide insight into the factors contributing to the observed spikes in variability.

Examining the distribution of the 128 CVerr values for each setting across the three designs reveals that settings 10 and 22 exhibit notably higher variability, particularly in CCV128x0 and RCV128. These settings show a few outlier values significantly larger than the others, while in CCV8x4x4, this effect is much less pronounced. Table 10 indicates that settings 10 and 22 are characterized by the lowest bagging fraction and highest learning rate, differing only in the number of iterations. However, since not all CVerrs computed with settings 10 and 22 are substantially higher than those from other settings, it is likely that the unusual values are caused by the specific RBseeds used, rather than the hyperparameter values.

As noted earlier, our goal is not realistic tuning. However, we investigate the unusual CVerr value of 13.7% observed in the CCV128x0 design for setting 22, with CVseed = 8 and Sseed = Fseed = 190.

Figure 8 shows the CVerr values for all settings on the Penguin dataset using the partition created with CVseed = 8. The CVerr values obtained with Sseed = Fseed = 190 are indicated by blue triangles, while those obtained with random RBseeds are represented

Table 10 LGB: CVerrs for the 8th block of CCV/128 and 6th repetition of RCV/128

Settings		CCV/128x0 8th seed				RCV/128 6th repetition			
SettingNo	Bagging fraction	Learning rate	Mum iterations	Seeds		CVerr (%)		Seeds	
				CVseeds	Fseeds	CVseeds	Fseeds	CVseeds	Fseeds
1	0.5	0.05	50	8	169	2.3	15481	121	15481
2	0.75	0.05	50	8	170	2.0	15482	122	15482
3	1	0.05	50	8	171	2.6	15483	123	15483
4	0.5	0.1	50	8	172	1.5	15484	124	15484
5	0.75	0.1	50	8	173	1.5	15485	125	15485
6	1	0.1	50	8	174	2.0	15486	126	15486
7	0.5	0.3	50	8	175	1.8	15487	127	15487
8	0.75	0.3	50	8	176	1.8	15488	128	15488
9	1	0.3	50	8	177	2.0	15489	129	15489
10	0.5	0.5	50	8	178	1.2	15490	130	15490
11	0.75	0.5	50	8	179	2.3	15491	131	15491
12	1	0.5	50	8	180	2.0	15492	132	15492
13	0.5	0.05	120	8	181	1.8	15493	133	15493
14	0.75	0.05	120	8	182	1.8	15494	134	15494
15	1	0.05	120	8	183	1.8	15495	135	15495
16	0.5	0.1	120	8	184	2.0	15496	136	15496
17	0.75	0.1	120	8	185	1.8	15497	137	15497
18	1	0.1	120	8	186	2.3	15498	138	15498
19	0.5	0.3	120	8	187	1.8	15499	139	15499
20	0.75	0.3	120	8	188	2.3	15500	140	15500
21	1	0.3	120	8	189	2.6	15501	141	15501
22	0.5	0.5	120	8	190	13.7	15502	142	15502
23	0.75	0.5	120	8	191	2.0	15503	143	15503
24	1	0.5	120	8	192	2.0	15504	144	15504

Unusually large CVerr values are highlighted in bold face

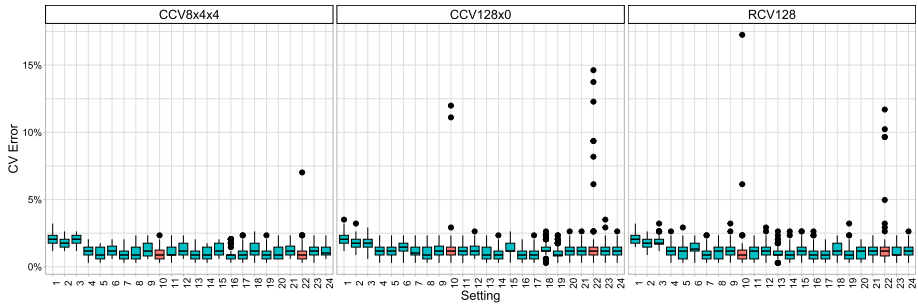


Fig. 7 LGB: distribution of the individual CVerr obtained for each one of the 24 settings considered for all designs with 128 blocks or repetitions. Highlighted in red are the box-plots for settings number 10 and 22 (Color figure online)

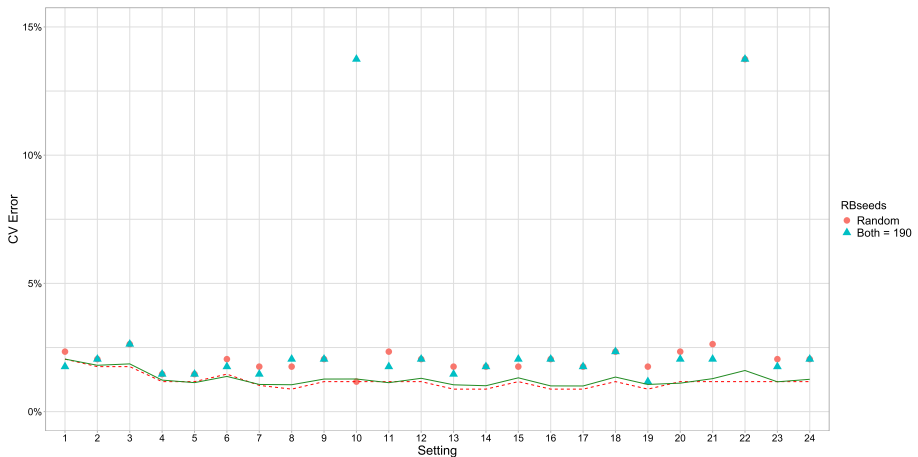


Fig. 8 LGB: CVerr values for all settings on the Penguin dataset using CVseed = 8. Red circles represent CVerr values with Sseed = Fseed = 190, while blue triangles show values with random RBseeds. The red solid line indicates the average CVerr, and the green dotted line marks the median CVerr from the CCV128x0 design (Color figure online)

by red circles (see Table 10). The solid red line and dotted green line indicate the average and median CVerr values, respectively, across all 128 CVseeds in the CCV128x0 design. While most CVerr values for CVseed = 8 are higher than the overall CCV128x0 averages, only setting 22 exhibits a significantly large CVerr, reinforcing the suspicion that this unusual result may be caused by the specific Sseed and Fseed, both set to 190 in this case.

The heatmap in Fig. 9 presents the CVerr values computed with CVseed = 8, while Fseed and Sseed range from 186 to 195. The numerical CVerr values are displayed inside the cells, with darker colors corresponding to higher values. The plot clearly shows areas of unusually high CVerr values, particularly at specific Fseed and Sseed combinations, such as (190, 191), (191, 191), and (190, 190), where the CVerr reaches 18.1%, 13.7%, and 12.6%, respectively.

These areas of high CVerr values agree with observations from previous analyses. The anomalies arise only for specific seed combinations, when most other combinations



Fig. 9 LGB: CVerr values for setting number 22 obtained on the CV partition created with initial seed CVseed = 8, and Seeds and Fseeds in the range 186 to 195

produce CVerr values below 3%, consistent with the overall trend for this design. This highlights that the interaction between Fseed and Sseed plays a critical role, as the seeds themselves are not inherently problematic. This finding emphasizes the need to account for such unpredictable interactions when studying a learner's predictive performance.

5 Discussion

The findings presented in this study demonstrate the significant advantages of blocking strategies in controlling cross-validation variability when studying a learner's predictive performance. In our study focussed on Grid Search for hyperparameter tuning, we found that CCV consistently outperforms RCV across multiple datasets and learning algorithms. Theoretical support for the method further validates its reliability and robustness. Notably, CCV achieves a lower CVstderr than RCV, especially at low run counts where CVstderr is relatively large, making it the preferable choice in such scenarios to reduce variability and improve the reliability of performance estimates.

A key benefit of CCV is its ability to reduce CVstderr by isolating and managing random effects, providing greater stability and precision in estimates. CCV is beneficial for

both deterministic learners, such as k-Nearest Neighbors, and stochastic learners, including Gradient Boosting Machines and Random Forests. When multiple initial seeds can be set, blocking for all of them further minimizes variability, leading to more consistent results. This approach is critical for achieving reproducibility and interpretability, particularly for complex learners.

The advantages of controlling random behavior extend beyond traditional grid searches to more advanced hyperparameter tuning methods, such as adaptive or Bayesian optimization. In these methods, uncontrolled random variability can obscure signals from prior evaluations, diminishing their efficiency and accuracy. Incorporating blocking strategies into such approaches enhances reliability and effectiveness. Furthermore, CCV shows promise for more sophisticated machine learning methods, such as reinforcement learning and neural networks, where random factors significantly influence optimization processes.

Applying CCV instead of RCV improves computational efficiency by reducing the number of data splits required, without incurring additional cost. This practical advantage makes CCV an appealing alternative to RCV in most scenarios, offering better control over variability and improved experimental design.

The empirical results also highlight the importance of managing interactions between random seeds. Specific seed combinations can lead to unusually high variability, as observed in the Penguin dataset, where certain configurations produced outlier CVerr values. These findings demonstrate that while individual seeds may not inherently cause instability, their combined effects can produce unpredictable outcomes. This reinforces the need for a nuanced approach to random seed management during hyperparameter tuning, particularly in high-stakes applications.

In summary, CCV provides a robust framework for managing cross-validation variability, offering advantages for both deterministic and stochastic learners. Supported by theoretical and empirical evidence, CCV is a promising candidate for standard use in experimental design. Its integration into advanced optimization strategies and sophisticated machine learning techniques further broadens its applicability and highlights its potential for future research.

6 Concluding remarks

This study demonstrates the efficacy of Controlled Cross-Validation (CCV) in reducing the variability of cross-validation error estimates, thereby enabling more precise comparisons of hyperparameter settings. The proposed blocking strategies effectively mitigate the influence of both CV partitioning and learners' random behavior, providing a more accurate evaluation of different hyperparameter settings compared to RCV. Importantly, this improved precision comes at no additional cost and even enhances computational efficiency. Given these advantages, CCV should always be preferred over RCV for hyperparameter tuning.

The theoretical underpinnings of CCV were validated through empirical examples involving diverse datasets and learning algorithms. The results consistently favored CCV designs, which achieved lower CVstderr compared to RCV designs. These findings underscore the importance of incorporating controlled blocking into hyperparameter tuning workflows, especially for stochastic learners where random effects can obscure meaningful differences between settings.

Looking ahead, the integration of CCV into mainstream hyperparameter tuning frameworks could significantly enhance the reproducibility and interpretability of machine learning experiments. Future research should focus on optimizing CCV for large-scale applications and exploring its interactions with adaptive search strategies.

By addressing a critical gap in hyperparameter tuning methodology, this study establishes CCV as a promising approach for experimental design, paving the way for more reliable and efficient machine learning workflows.

Moreover, the advantages of CCV are also valuable in sensitivity analysis. While in this study, hyperparameter values were treated as nominal levels to accommodate non-linear behaviors without delving into the complexities of sensitivity analysis, it is worth noting that blocking still enhances the precision of analysis when employing CVerr for fine-tuning a learner. An intriguing possibility is to implement blocking with respect to CV partitioning using second-order rotatable designs, as proposed in Lujan-Moreno et al. (2018).

CCV can also play an important role in model selection. By enabling a fair comparison among different predictive learners, it adheres to the fundamental principle of “comparing like with like” within the blocking framework, as outlined in the minimization in Equation (4) of Bergstra and Bengio (2012), for example. This ensures that differences in predictive performance can be attributed solely to the learners themselves, rather than variability introduced by uncontrolled factors.

Appendix

Data description

The six small to medium size datasets used called for either classification or regression tasks, and in one case both were applied. Specifications of these datasets are provided in Table 2.

See Table 11.

Table 11 Data sets used in the examples

Dimensions			Task	
Dataset	Cases	Features	Classification	Regression
Sonar	208	60	Binary	
Penguins	342	5	3 Classes	
Boston	506	13		✓
Insurance	1338	6		✓
Wine	4898	11	7 Classes	✓
Adults	30162	13	Binary	

Sonar data

This dataset (Gorman & Sejnowski, 1988) contains 60 frequency readings from 208 objects, which can be classified as cylindrical rocks or mines. The task is to classify the objects into either rocks or mines.

Penguin data

This dataset (Gorman et al., 2014) includes 302 instances, each with five characteristics of penguins from three different species. The task is to classify the penguins into their respective species. This classification task is generally straightforward, with learners typically achieving a low classification error.

Boston crime data

This dataset (Harrison & Rubinfeld, 1978) contains 13 housing features from 506 households in various suburbs. The task is to predict the percentage rate of crime in these suburbs.

Insurance data

This dataset (Lantz, 2019) includes six characteristics from 1,338 health insurance beneficiaries. The task is to predict the medical costs billed by health insurance companies.

Wine data

This dataset (Cortez et al., 2009) contains 11 attributes from 4,898 variants of white wine. The task is to classify the wines into one of seven quality classes (rated on a Likert scale from three to nine). Additionally, a regression analysis was conducted on the target variable, as recommended by the authors, and results for both classification and regression tasks are presented.

Adult data

This dataset (Kohavi, 1996) includes 13 socio-economic features from 30,162 adults. The task is to classify individuals into either a low or high-income class.

Numerical tables

This section presents the ANOVA tables and lists the top 5 settings that yielded the lowest CVerr values across all designs and datasets.

k-NN

See Table 12.

Table 13 shows the standard errors of the mean CVerr, their ratio and the p -value of the CVseeds effect for increasing number of blocks or repetitions relative to all datasets.

See Tables 14 and 15.

Table 12 k-NN: ANOVA tables for the CVerr obtained with CCV12 and RCV12 designs for all datasets

	CCV12					RCV12			
	Df	SSE	MSE	Prob		Df	SSE	MSE	Prob
<i>Sonar</i>									
Settings	7	0.018331	0.0026187	0	7	0.023046	0.003292	0	
CVseeds	11	0.004693	0.0004266	0					
Residuals	77	0.011235	0.0001459		95	0.046967	0.000494		
<i>Penguins</i>									
Settings	7	0.007153	0.001022	0	7	0.010899	0.001557	0	
CVseeds	11	0.018155	0.001650	0					
Residuals	77	0.004222	0.000055		95	0.030724	0.000323		
<i>Boston</i>									
Settings	7	0.566779	0.0809684	0	7	0.783508	0.11193	0	
CVseeds	11	0.787411	0.0715829	0					
Residuals	77	0.309004	0.004013		95	1.84633	0.019435		
<i>Insurance</i>									
Settings	7	14094.09	2013.4412	0	7	13982.34	1997.478	0	
CVseeds	11	467.6597	42.514519	0					
Residuals	77	195.1849	2.5348692		95	14548.83	153.1456		
<i>Wine reg</i>									
Settings	7	0.020544	0.0029349	0	7	0.023387	0.003341	0	
CVseeds	11	0.001972	0.0001793	0					
Residuals	77	0.001583	0.0000206		95	0.028569	0.000301		
<i>Wine class</i>									
Settings	7	0.053928	0.00770	0	7	0.052926	0.007561	0	
CVseeds	11	0.000524	0.00005	0					
Residuals	77	0.000317	0.00000		95	0.053677	0.000565		
<i>Adults</i>									
Settings	7	0.01668	0.0023829	0	7	0.016529	0.002361	0	
CVseeds	11	0.00009	8.14E−06	0					
Residuals	77	0.00007	8.56E−07		95	0.016652	0.000175		

Table 13 k-NN: Standard errors for increasing number of blocks or repetitions for all datasets

Nblocks	CCV12	RCV12	CCV/RCV	p-value
<i>Sonar</i>				
2	0.011128	0.012139	0.916698	0.35
3	0.0095	0.009065	1.047957	0.83
4	0.007346	0.006909	1.0633	0.06
5	0.006098	0.005612	1.086556	0.05
6	0.005472	0.004983	1.098022	0.06
7	0.004687	0.004941	0.948708	0.01
8	0.004393	0.004952	0.887104	0.02
9	0.004155	0.004887	0.85022	0.01
10	0.003872	0.004437	0.872734	0.00
11	0.003734	0.004856	0.768906	0.00
12	0.003487	0.004759	0.73263	0.00
<i>Boston</i>				
2	0.033957	0.113363	30%	0.00
3	0.040884	0.070832	58%	0.00
4	0.037393	0.070853	53%	0.00
5	0.032128	0.057912	55%	0.00
6	0.028384	0.049435	57%	0.00
7	0.025864	0.043026	60%	0.00
8	0.024543	0.039165	63%	0.00
9	0.022773	0.036504	62%	0.00
10	0.021161	0.035516	60%	0.00
11	0.019484	0.033065	59%	0.00
12	0.018287	0.031725	58%	0.00
<i>Wine reg</i>				
2	0.001485	0.004227	35%	0.69
3	0.002731	0.003706	74%	0.17
4	0.002246	0.003031	74%	0.00
5	0.001806	0.00331	55%	0.00
6	0.001622	0.00299	54%	0.00
7	0.001476	0.002575	57%	0.00
8	0.001405	0.002735	51%	0.00
9	0.001489	0.002572	58%	0.00
10	0.001379	0.002563	54%	0.00
11	0.001336	0.002351	57%	0.00
12	0.001309	0.002215	59%	0.00
<i>Adults</i>				
2	0.000371	0.000799	46%	0.00
3	0.000463	0.00093	50%	0.00
4	0.000425	0.000799	53%	0.00
5	0.000361	0.00064	56%	0.00
6	0.000379	0.000556	68%	0.00
7	0.00039	0.000478	82%	0.00
8	0.000349	0.00043	81%	0.00
9	0.00032	0.000392	82%	0.00
10	0.000292	0.000374	78%	0.00

Table 13 (continued)

Nblocks	CCV12	RCV12	CCV/RCV	<i>p</i> -value
11	0.000285	0.000368	78%	0.00
12	0.000267	0.000341	78%	0.00
<i>Penguins</i>				
2	0.003473	0.011581	30%	0.00
3	0.002644	0.011543	23%	0.00
4	0.002095	0.008555	24%	0.00
5	0.001989	0.006976	29%	0.00
6	0.001907	0.007058	27%	0.00
7	0.001692	0.006369	27%	0.00
8	0.002217	0.00594	37%	0.00
9	0.002045	0.005343	38%	0.00
10	0.002067	0.004901	42%	0.00
11	0.002291	0.004505	51%	0.00
12	0.002138	0.004333	49%	0.00
<i>Insurance</i>				
2	0.957536	1.695755	56%	0.01
3	0.664884	1.69162	39%	0.00
4	0.643636	1.410999	46%	0.00
5	0.64957	1.192319	54%	0.00
6	0.645334	1.108863	58%	0.00
7	0.612406	0.985756	62%	0.00
8	0.546744	0.933277	59%	0.00
9	0.514683	0.855469	60%	0.00
10	0.517757	0.811325	64%	0.00
11	0.484974	0.788541	62%	0.00
12	0.459608	0.732427	63%	0.00
<i>Wine class</i>				
2	0.00156	0.001689	92%	0.00
3	0.001102	0.001649	67%	0.00
4	0.000924	0.001412	65%	0.00
5	0.001038	0.001131	92%	0.00
6	0.000903	0.001122	80%	0.00
7	0.00079	0.001097	72%	0.00
8	0.000771	0.001054	73%	0.00
9	0.000706	0.000987	71%	0.00
10	0.000645	0.000933	69%	0.00
11	0.000604	0.000886	68%	0.00
12	0.000585	0.000844	69%	0.00

The first columns shows the number of blocks or repetitions, next two columns show the stderr of the mean CVerr for CCV12 and RCV12, respectively, the fourth column their percent ratio and the fifth the *p*-value of the CVseeds effects

Table 14 k-NN: Ordered CVerr values for the full 12 blocks or repetitions Designs, four smallest datasets

CCV12				RCV12			
k	Dist	SettingNo	C $\bar{V}$ err	k	Dist	SettingNo	C $\bar{V}$ err
<i>Sonar</i>							
5	L1	3	0.185	5	L1	3	0.185
4	L1	2	0.187	3	L2	5	0.190
3	L2	5	0.188	6	L1	4	0.191
3	L1	1	0.190	3	L1	1	0.192
6	L1	4	0.195	4	L1	2	0.193
4	L2	6	0.195	4	L2	6	0.196
5	L2	7	0.214	5	L2	7	0.215
6	L2	8	0.226	6	L2	8	0.234
<i>Penguins</i>							
3	L1	1	0.207	3	L1	1	0.205
5	L1	3	0.213	5	L1	3	0.213
5	L2	7	0.221	3	L2	5	0.220
3	L2	5	0.221	4	L1	2	0.223
4	L1	2	0.223	5	L2	7	0.227
6	L1	4	0.226	6	L1	4	0.229
4	L2	6	0.233	6	L2	8	0.237
6	L2	8	0.234	4	L2	6	0.239
<i>Boston</i>							
6	L1	4	2.12	6	L1	4	2.085
5	L1	3	2.17	5	L1	3	2.142
4	L1	2	2.23	4	L1	2	2.178
6	L2	8	2.25	6	L2	8	2.206
5	L2	7	2.28	5	L2	7	2.249
4	L2	6	2.31	4	L2	6	2.299
3	L2	5	2.34	3	L1	1	2.313
3	L1	1	2.35	3	L2	5	2.376
<i>Insurance</i>							
6	L1	4	103.9	6	L1	4	104.0
5	L1	3	104.7	5	L1	3	104.5
4	L1	2	106.4	4	L1	2	106.8
3	L1	1	110.1	3	L1	1	110.2
6	L2	8	127.8	6	L2	8	127.1
5	L2	7	128.3	5	L2	7	128.0
4	L2	6	129.8	4	L2	6	130.1
3	L2	5	134.2	3	L2	5	134.6

Table 15 k-NN: ordered C $\bar{V}$ err for the full 12 blocks or repetitions designs, two largest datasets

CCV12				RCV12			
k	Dist	SettingNo	C $\bar{V}$ err	k	Dist	SettingNo	C $\bar{V}$ err
<i>Wine reg</i>							
6	L1	4	0.6393	6	L1	4	0.6373
5	L1	3	0.6429	5	L1	3	0.6412
4	L1	2	0.6527	4	L1	2	0.6500
6	L2	8	0.6601	6	L2	8	0.6612
5	L2	7	0.6633	5	L2	7	0.6634
3	L1	1	0.6682	3	L1	1	0.6659
4	L2	6	0.6718	4	L2	6	0.6726
3	L2	5	0.6870	3	L2	5	0.6878
<i>Wine class</i>							
3	L2	5	0.8511	3	L2	5	0.8518
3	L1	1	0.8525	3	L1	1	0.8523
4	L2	6	0.8825	4	L2	6	0.8828
4	L1	2	0.8848	4	L1	2	0.8829
5	L2	7	0.9016	5	L1	3	0.9005
5	L1	3	0.9020	5	L2	7	0.9020
6	L2	8	0.9146	6	L1	4	0.9138
6	L1	4	0.9152	6	L2	8	0.9154
<i>Adults</i>							
6	L1	4	0.2124	6	L1	4	0.2125
4	L1	2	0.2151	4	L1	2	0.2154
6	L2	8	0.2156	6	L2	8	0.2157
4	L2	6	0.2187	4	L2	6	0.2186
5	L1	3	0.2282	5	L1	3	0.2286
5	L2	7	0.2329	5	L2	7	0.2331
3	L1	1	0.2436	3	L1	1	0.2436
3	L2	5	0.2499	3	L2	5	0.2498

Random forests

See Tables 16, 17 and 18.

Table 16 RF: ANOVA tables for the CVerr obtained with CCV8x4, CCV32x0 and RCV32 designs for the four smallest datasets

	CCV8x4			CCV32x0			RCV32		
	Df	MSE	Prob	Df	MSE	Prob	Df	MSE	Prob
<i>Sonar</i>									
SettingNo	83	0.006663	0	83	0.006357	0	83	0.006503	0
CVseeds	7	0.026149	0						
RBseeds	3	0.000414	0.119	31	0.011777	0			
Residuals	2594	0.000136		2573	0.000124		2604	0.000299	
Tot	2687	0.000406		2687	0.000451		2687	0.000491	
<i>Penguins</i>									
SettingNo	83	0.000921	0	83	0.000971	0	83	0.001036	0
CVseeds	7	0.004826	0						
RBseeds	3	8.13E−06	0.355	31	0.001185	0			
Residuals	2594	8.10E−06		2573	9.75E−06		2604	2.14E−05	
Tot	2687	4.88E−05		2687	5.30E−05		2687	5.27E−05	
<i>Boston</i>									
SettingNo	83	0.153528	0	83	0.162779	0	83	0.137599	0
CVseeds	7	2.870134	0						
RBseeds	3	0.001665	0.412	31	0.746161	0			
Residuals	2594	0.00074		2573	0.001276		2604	0.005395	
Tot	2687	0.012936		2687	0.014859		2687	0.009479	
<i>Insurance</i>									
SettingNo	83	42.59767	0	83	41.05916	0	83	40.71765	0
CVseeds	7	10.45908	0						
RBseeds	3	0.099455	0.002	31	3.878797	0			
Residuals	2594	0.026444		2573	0.03339		2604	0.0957	
Tot	2687	1.368707		2687	1.345019		2687	1.350491	

Table 17 RF: ANOVA tables for the CVerr obtained with CCV8x4, CCV32x0 and RCV32 designs for the two larger datasets

	CCV8x4			CCV32x0			RCV32		
	Df	MSE	Prob	Df	MSE	Prob	Df	MSE	Prob
<i>Wine reg</i>									
SettingNo	83	0.011028	0	83	0.011156	0	83	0.011239	0
CVseeds	7	0.001888	0						
RBseeds	3	2.19E-06	0.031	31	0.000783	0			
Residuals	2594	6.84E-07		2573	1.35E-06		2604	1.02E-05	
Tot	2687	0.000346		2687	0.000355		2687	0.000357	
<i>Wine class</i>									
SettingNo	83	0.013342	0	83	0.013263	0	83	0.012936	0
CVseeds	7	0.00145	0						
RBseeds	3	7.35E-06	0.279	31	0.000647	0			
Residuals	2594	5.53E-06		2573	5.78E-06		2604	1.61E-05	
Tot	2687	0.000421		2687	0.000423		2687	0.000415	
<i>Adults</i>									
SettingNo	83	0.000908	0	83	0.000939	0	83	0.000933	0
CVseeds	7	0.000141	0						
RBseeds	3	6.51E-08	0.993	31	3.21E-05	0			
Residuals	2594	3.30E-07		2573	3.83E-07		2604	7.36E-07	
Tot	2687	2.87E-05		2687	2.98E-05		2687	2.95E-05	

Table 18 RF: Five settings that yielded the smallest CV $\bar{V}err$ values for all datasets and designs

CCV3x4		CCV32x0					RCV32				
		min node size	rplic	sample frac-tion	Sett.	CV $\bar{V}err$	min node size	rplic	sample frac-tion	Sett.	CV $\bar{V}err$
<i>Sonar</i>											
5	5	F	0.9	64	0.155	5	3	F	0.9	61	0.152
5	3	F	0.9	61	0.156	5	5	F	0.9	64	0.155
10	5	F	0.9	65	0.161	10	3	F	0.9	62	0.155
10	3	F	0.9	62	0.162	10	5	F	0.9	37	0.160
5	10	F	0.9	67	0.163	5	3	F	0.9	65	0.161
<i>Penguins</i>											
2	1	F	0.7	37	0.015	2	1	F	0.7	37	0.013
2	1	T	1	73	0.015	2	3	T	0.9	52	0.014
2	1	T	0.9	49	0.015	2	1	F	0.9	61	0.015
2	3	T	1	76	0.016	2	1	T	0.9	49	0.015
2	3	T	0.9	52	0.016	2	3	T	1	76	0.015
<i>Boston</i>											
3	15	T	1	82	1.76	3	15	T	0.9	58	1.78
3	15	F	0.7	46	1.77	3	15	T	1	82	1.78
3	10	F	0.5	19	1.77	3	15	F	0.7	46	1.78
3	10	T	1	79	1.77	3	10	F	0.7	43	1.78
3	10	T	0.7	31	1.77	3	10	T	0.9	55	1.78
4	15	T	0.5	11	20.9	4	15	T	0.5	11	20.8
4	15	F	0.5	23	21.0	4	15	F	0.5	23	20.9
4	10	T	0.5	8	21.0	4	10	T	0.5	8	21.0
4	15	T	0.7	35	21.1	4	15	T	0.7	35	21.0
6	15	T	0.5	12	21.1	6	15	T	0.5	12	21.1

Table 18 (continued)

CCV8x4						CCV32x0						RCV32					
entry	min node size	rp	sample frac-tion	Sett.	CV̄err	entry	min node size	rp	sample frac-tion	Sett.	CV̄err	entry	min node size	rp	sample frac-tion	Sett.	CV̄err
Wine reg																	
3	3	F	0.9	61	0.350	3	3	F	0.9	61	0.350	3	3	F	0.9	61	0.349
3	5	F	0.9	64	0.354	3	5	F	0.9	64	0.354	3	5	F	0.9	64	0.353
6	3	F	0.9	62	0.358	6	3	F	0.9	62	0.357	6	3	F	0.9	62	0.357
3	3	F	0.7	37	0.361	3	3	F	0.7	37	0.360	3	3	F	0.7	37	0.361
6	5	F	0.9	65	0.361	6	5	F	0.9	65	0.360	6	5	F	0.9	65	0.361
Wine class																	
3	3	T	1	73	0.310	3	3	T	0.9	49	0.311	3	5	T	1	76	0.310
3	3	T	0.9	49	0.310	3	3	T	1	73	0.311	3	3	T	0.9	49	0.311
3	5	T	1	76	0.310	3	5	T	1	76	0.311	3	3	T	1	73	0.311
3	3	F	0.7	37	0.311	3	3	F	0.7	37	0.312	3	3	T	0.7	25	0.311
3	5	T	0.9	52	0.313	3	3	T	0.7	25	0.313	3	3	F	0.7	37	0.312
Adults																	
3	3	T	0.7	28	0.137	3	10	F	0.5	22	0.137	3	5	T	0.7	31	0.136
3	10	F	0.5	22	0.137	3	3	T	0.5	4	0.137	3	3	T	0.5	4	0.136
3	5	T	0.5	7	0.137	3	10	T	0.5	10	0.137	3	10	T	0.7	34	0.136
3	10	T	0.9	58	0.137	3	5	T	0.5	7	0.137	3	10	F	0.5	22	0.136
3	10	T	0.5	10	0.137	3	10	T	0.7	34	0.137	3	3	T	0.7	28	0.137

Light gradient boosting machines

See Tables 19, 20 and 21.

Table 19 LGB: ANOVA tables for the CVerr obtained with CCV8x4x4, CCV128x0 and RCV128 designs for the four smallest datasets

	CCV 8x4x4						CCV 128x0						RCV 128					
	Df		SSE	MSE*	Prob		Df		SSE	MSE*	Prob		Df		SSE	MSE*		Prob
<i>Sonar</i>																		
Settings	23		1.1933	0.0519	0.00		23		1.2206	0.0531	0.00		23		1.1387	0.0495		0.00
CVseeds	7		0.2351	0.0336	0.00		127		0.3540	0.0028	0.00							
Sseeds	3		0.0045	0.0015	0.00													
Fseeds	3		0.0039	0.0013	0.00													
Residuals	3035		0.6095	0.0002			2921		0.6809	0.0002			3048		1.0706	0.0004		
Total	3071		2.0464	0.0007			3071		2.2555	0.0007			3071		2.2093	0.0007		
<i>Penguins</i>																		
Settings	23		0.029	0.00127	0.00		23		0.024	0.00103	0.00		23		0.024	0.00105		0.00
CVseeds	7		0.046	0.00661	0.00		127		0.041	0.00033	0.00							
Sseeds	3		0.000	0.00002	0.14													
Fseeds	3		0.000	0.00003	0.01													
Residuals	3035		0.030	0.00001			2921		0.110	0.00004			3048		0.126	0.00004		
Total	3071		0.105	0.00003			3071		0.175	0.00006			3071		0.150	0.00005		
<i>Boston</i>																		
Settings	23		245.98	10.69	0.00		23		263.49	11.46	0.00		23		267.83	11.64		0.00
CVseeds	7		69.53	9.93	0.00		127		31.81	0.25	0.00							
Sseeds	3		0.02	0.01	1.00													
Fseeds	3		0.19	0.06	0.00													
Residuals	3035		26.67	0.01			2921		21.06	0.01			3048		46.40	0.02		
Total	3071		342.38	0.11			3071		316.37	0.10			3071		314.23	0.10		

Table 19 (continued)

	CCV 8x4x4				CCV 128x0				RCV128			
	Df	SSE	MSE*	Prob	Df	SSE	MSE*	Prob	Df	SSE	MSE*	Prob
<i>Insurance</i>												
Settings	23	494878	21516	0.00	23	571404	24844	0.00	23	535080	23264	0.00
CVseeds	7	20359	2908	0.00	127	15139	119	0.00				
Sseeds	3	33	11	0.06								
Fseeds	3	23846	7949	0.00								
Residuals	3035	16074	5		2921	14197	5		3048	27222	9	
Total	3071	555190	181		3071	600740	196		3071	562301	183	

*Values multiplied by 1000

Table 20 LGB: ANOVA tables for the CVerr obtained with CCV8x4x4, CCV128x0 and RCV128 designs for the two larger datasets

	CCV 8x4x4				CCV 128x0				RCV128			
	Df	SSE	MSE*	Prob	Df	SSE	MSE*	Prob	Df	SSE	MSE*	Prob
<i>Wine reg</i>												
Settings	23	2.007	0.08727	0.00	23	2.057	0.08944	0.00	23	2.044	0.08887	0.00
CV/seeds	7	0.015	0.00211	0.00	127	0.021	0.00017	0.00				
Sseeds	3	0.000	0.00003	0.07								
Fseeds	3	0.000	0.00005	0.00								
Residuals	3035	0.058	0.00002		2921	0.063	0.00002		3048	0.089	0.00003	
Total	3071	2.080	0.00068		3071	2.141	0.00070		3071	2.133	0.00069	
<i>Wine class</i>												
Settings	23	25.029	1.0882	0.00	23	25.523	1.1097	0.00	23	26.294	1.1432	0.00
CV/seeds	7	0.019	0.0027	0.00	127	0.084	0.0007	0.24				
Sseeds	3	0.001	0.0004	0.37								
Fseeds	3	0.016	0.0054	0.00								
Residuals	3035	1.933	0.0006		2921	1.804	0.0006		3048	1.937	0.0006	
Total	3071	26.999	0.0088		3071	27.410	0.0089		3071	28.231	0.0092	
<i>Adults</i>												
Settings	23	0.05831	2.53523	0.00	23	0.05751	2.50029	0.00	23	0.05826	2.53314	0.00
CV/seeds	7	0.00019	0.02720	0.00	127	0.00029	0.00228	0.00				
Sseeds	3	0.00000	0.00020	1.00								
Fseeds	3	0.00001	0.00201	0.00								
Residuals	3035	0.00202	0.00067		2921	0.00196	0.00067		3048	0.00234	0.00077	
Total	3071	0.06053	0.01971		3071	0.05976	0.01946		3071	0.06060	0.01973	

*Values multiplied by 1000

Table 21 LGB: Five settings that yielded the smallest CV $\bar{e}$ r values for all datasets and designs

CCV128x4										RCV128									
CCV128x0										RCV128									
Bagging fraction	Learning rate	Num iterations	SettingNo	CV $\bar{e}$ r	Bagging fraction	Learning rate	Num iterations	SettingNo	CV $\bar{e}$ r	Bagging fraction	Learning rate	Num iterations	SettingNo	CV $\bar{e}$ r	Bagging fraction	Learning rate	Num iterations	SettingNo	CV $\bar{e}$ r
<i>Sonar</i>																			
1	0.3	120	21	0.1387	1	0.3	120	21	0.1329	1	0.3	120	21	0.1332			120	21	0.1332
1	0.5	120	24	0.1397	1	0.5	120	24	0.1362	1	0.5	50	12	0.1402			50	12	0.1402
0.75	0.3	120	20	0.1406	0.75	0.3	120	20	0.1381	1	0.5	120	24	0.1407			120	24	0.1407
0.75	0.1	120	17	0.1423	1	0.5	50	12	0.1388	0.75	0.1	120	17	0.1423			120	17	0.1423
1	0.5	50	12	0.1447	0.75	0.1	120	17	0.1397	1	0.3	50	9	0.1426			50	9	0.1426
<i>Penguins</i>																			
0.5	0.1	120	16	0.0090	0.75	0.1	120	17	0.0100	0.75	0.1	120	17	0.0098			120	17	0.0098
0.5	0.3	120	19	0.0091	0.5	0.1	120	16	0.0100	0.5	0.1	120	16	0.0099			120	16	0.0099
0.5	0.05	120	13	0.0093	0.75	0.05	120	14	0.0101	0.5	0.3	50	7	0.0101			50	7	0.0101
0.75	0.1	120	17	0.0094	0.5	0.05	120	13	0.0105	0.5	0.3	120	19	0.0101			120	19	0.0101
0.5	0.3	50	7	0.0095	0.75	0.3	50	8	0.0105	0.5	0.05	120	13	0.0101			120	13	0.0101
<i>Boston</i>																			
0.5	0.05	120	13	1.844	0.5	0.05	50	1	1.845	0.5	0.05	50	1	1.845			50	1	1.845
0.5	0.1	50	4	1.844	0.5	0.05	120	13	1.846	0.5	0.1	50	4	1.846			50	4	1.846
0.5	0.05	50	1	1.851	0.5	0.1	50	4	1.851	0.5	0.05	120	13	1.846			120	13	1.846
0.75	0.05	50	2	1.865	0.75	0.05	50	2	1.864	0.75	0.05	50	2	1.861			50	2	1.861
0.75	0.1	50	5	1.911	0.75	0.1	50	5	1.916	1	0.05	50	3	1.921			50	3	1.921
<i>Insurance</i>																			
0.5	0.05	50	1	134.8	0.5	0.05	50	1	133.1	0.5	0.05	50	1	133.8			50	1	133.8
0.75	0.05	50	2	135.6	1	0.05	50	3	134.1	1	0.05	50	3	135.1			50	3	135.1
1	0.05	50	3	135.9	0.5	0.05	120	13	136.4	0.75	0.05	50	2	135.6			50	2	135.6
0.5	0.1	50	4	137.4	0.75	0.05	50	2	136.6	0.5	0.05	120	13	137.0			120	13	137.0
0.5	0.05	120	13	137.9	0.75	0.1	50	5	137.0	0.5	0.1	50	4	137.3			50	4	137.3

Table 21 (continued)

CCV8x4x4										CCV128x0										RCV128									
Bagging fraction	Learning rate	Num iterations	SettingNo	CVerr	Bagging fraction	Learning rate	Num iterations	SettingNo	CVerr	Bagging fraction	Learning rate	Num iterations	SettingNo	CVerr	Bagging fraction	Learning rate	Num iterations	SettingNo	CVerr	Bagging fraction	Learning rate	Num iterations	SettingNo	CVerr	Bagging fraction	Learning rate	Num iterations	SettingNo	CVerr
<i>Wine reg</i>																													
1	0.3	120	21	0.4336	1	0.3	120	21	0.4326	1	0.3	120	21	0.4315	1	0.3	120	21	0.4304	1	0.3	120	21	0.4293	1	0.3	120	21	0.4282
1	0.1	120	18	0.4407	1	0.1	120	18	0.4397	1	0.1	120	18	0.4386	1	0.1	120	18	0.4375	1	0.1	120	18	0.4364	1	0.1	120	18	0.4353
0.75	0.1	120	17	0.4416	0.75	0.1	120	17	0.4407	0.75	0.1	120	17	0.4397	0.75	0.1	120	17	0.4386	0.75	0.1	120	17	0.4375	0.75	0.1	120	17	0.4364
0.5	0.1	120	16	0.4464	0.5	0.1	120	16	0.4454	0.5	0.1	120	16	0.4443	0.5	0.1	120	16	0.4432	0.5	0.1	120	16	0.4421	0.5	0.1	120	16	0.4410
0.75	0.3	120	20	0.4470	0.5	0.1	120	16	0.4464	0.5	0.1	120	16	0.4453	0.5	0.1	120	16	0.4442	0.5	0.1	120	16	0.4431	0.5	0.1	120	16	0.4420
<i>Wine class</i>																													
1	0.1	120	18	0.3535	1	0.1	120	18	0.3524	1	0.1	120	18	0.3513	1	0.1	120	18	0.3502	1	0.1	120	18	0.3491	1	0.1	120	18	0.3480
0.75	0.1	120	17	0.3552	0.75	0.1	120	17	0.3542	0.75	0.1	120	17	0.3531	0.75	0.1	120	17	0.3520	0.75	0.1	120	17	0.3509	0.75	0.1	120	17	0.3498
0.5	0.1	120	16	0.3622	0.5	0.1	120	16	0.3612	0.5	0.1	120	16	0.3601	0.5	0.1	120	16	0.3590	0.5	0.1	120	16	0.3579	0.5	0.1	120	16	0.3568
0.75	0.05	120	14	0.3763	0.75	0.05	120	14	0.3753	0.75	0.05	120	14	0.3742	0.75	0.05	120	14	0.3731	0.75	0.05	120	14	0.3720	0.75	0.05	120	14	0.3709
1	0.05	120	15	0.3770	1	0.05	120	15	0.3760	1	0.05	120	15	0.3749	1	0.05	120	15	0.3738	1	0.05	120	15	0.3727	1	0.05	120	15	0.3716
<i>Adults</i>																													
1	0.1	120	18	0.1303	1	0.1	120	18	0.1292	1	0.1	120	18	0.1281	1	0.1	120	18	0.1270	1	0.1	120	18	0.1259	1	0.1	120	18	0.1248
0.75	0.1	120	17	0.1305	0.75	0.1	120	17	0.1295	0.75	0.1	120	17	0.1284	0.75	0.1	120	17	0.1273	0.75	0.1	120	17	0.1262	0.75	0.1	120	17	0.1251
1	0.3	50	9	0.1309	1	0.3	50	9	0.1299	1	0.3	50	9	0.1288	1	0.3	50	9	0.1277	1	0.3	50	9	0.1266	1	0.3	50	9	0.1255
0.5	0.1	120	16	0.1315	0.5	0.1	120	16	0.1305	0.5	0.1	120	16	0.1294	0.5	0.1	120	16	0.1283	0.5	0.1	120	16	0.1272	0.5	0.1	120	16	0.1261
0.75	0.3	50	8	0.1320	0.75	0.3	50	8	0.1310	0.75	0.3	50	8	0.1299	0.75	0.3	50	8	0.1288	0.75	0.3	50	8	0.1277	0.75	0.3	50	8	0.1266

Survey of tuning or autotuning software

In this section we offer an overview of several popular software solutions available on the web or as packages for Python and R to run hyperparameter optimization automatically. Even though some of these use sophisticated search procedures, none of these offers options for controlling for CVseed or RBseed. In several cases initial seeds are not even mentioned or the documentation is sparse.

Software

- *Web and stand-alone apps*

- *SMAC*

The possibility of choosing ParamILS with algorithm's random seeds controlled seems not to be able in Version 3.

- *Google Vizier*

Neither on the Vizier website nor in a paper presenting Vizier there is any mention of controlling for the initial seeds.

- *Neptune*

It is a Python Library and web app to run experiments with ML algorithms. No mention for seed controlling in intro or best practice section. In advice about running CV explicitly mention CV partition and suggests using RCV.

- *Azure Machine Learning*

The hyperparameter tuning guide is not very informative and does not mention CV.

- *SageMaker*

In the automating tuning consideration page, it is stated that you can specify a single integer as a random seed for hyperparameter tuning, but there is no control for the seed. In the section, it is suggested to set a random seed for reproducibility purposes.

- *R and Python libraries*

- *caret* The *trainControl* function takes an optional list of indices that are used to create the resamples at each iteration. We are not sure about the implementation because “train” takes both simple “cv” option and “Repeated cv”. However, this library seems to allow for controlling the CV partition but it is not one of the available options.

- *mlr/mlrTuning*

Offers Grid-search as well as advanced adaptive search methods. There is no mention of controlling for initial seeds. In presenting Bayesian Optimization (BO) it is stated “If we replicate running our BO algorithm ten times (with random initial designs and varying random seeds) and compare this to a random search, we can see that BO indeed performs much better and on average reaches the global optimum after around 15 function evaluations. We presume that this means that the search is simply repeated using random seeds.”

- *hyperopt*

In “RandomizedSearchCV” from the *sklearn.model_selection* module. Starting seeds are not even mentioned.

- *h2o*
No reference to seed blocking for Grid Search. The option for specifying the random number generator (RNG) seed for algorithms that are dependent on randomization.
- *scikit-learn*
In the user guide for *GridSearchCV* random seeds are not mentioned. In the user guide for model selection, it is mentioned that “Prior to version *scipy 0.16* do not allow specifying a random state. However, beginning *scikit-learn 0.18*, the *sklearn.model_selection* module sets the random state provided by the user if *scipy* ≥ 0.16 is also available.” No mention of controlling for CV partitions.
- *Skiti-Optimize*
Also called *skopt*, replaces *scikit-learn*’s *GridSearchCV* with *BayesSearchCV*. No mention of options for setting seeds.
- *BayesianOptimization*
The seed for CV is neither mentioned in the documentation nor set in the example.
- *h2oAutoML*
There is no reference to seed setting neither in the documentation for GS nor in the examples within.
- *EZtune*
The *eztune_cv* function does not have an option for setting seeds and in the vignettes seeds are not set in the examples given in the vignettes.
- *tune*
Part of the *tidymodels* ecosystem for R. The *tune_grid* for tuning hyperparameters does not have an option for setting seeds and only a starting seed is set externally for the whole tuning in the examples therein.
- *mlmachine*
Sparse documentation, based on code snippets. There is no mention of setting initial seeds.
- *Polyaxon*
Easy-to-use, Scalability, Flexibility, Efficiency. Sparse documentation. The GS function “*V1GridSearch*” does not have an option for setting seeds. The random search function “*V1RandomSearch*” only allows setting one initial seed.
- *Talos*
Fully automating hyperparameter tuning and model evaluation. Only one starting seed can be set in the “Scan” function. No mention of controlling seeds.
- *GPyOpt*
Python package with rigorous approach to Bayesian optimization. It includes the possibility of using Latin square grids (designs). However, there is no mention of controlling for the initial seed when building the grid (designs).

Author contributions Only the author contributed to the manuscript.

Funding The work has not been funded in any way.

Data availability The raw data used for the examples are freely available as indicated in the manuscript. Edited data used for the examples are available upon request to the author.

Code availability R programs used for the examples are available upon request to the author.

Declarations

Conflict of interest The author has no conflicting financial or non-financial interests that are directly or indirectly related to this work.

Ethics approval This article does not require permission for ethics approval as the numerical examples are based on publicly available datasets.

Consent to participate This article does not require consent to participation as the numerical examples are based on publicly available datasets.

Consent for publication The author of this manuscript consents to the publication of the manuscript.

References

- Adenso-Díaz, B., & Laguna, M. (2006). Fine-tuning of algorithms using fractional experimental designs and local search. *Operations Research*, 54(1), 99–114.
- Allen, D. S. (1974). The relationship between variable selection and data augmentation and a method for prediction. *Technometrics*, 16(1), 125–127.
- Alpaydin, E. (2020). *Introduction to machine learning* (4th ed.). MIT Press.
- Arlot, S., & Celisse, A. (2010). A survey of cross-validation procedures for model selection. *Statistics Surveys*, 4, 40–7.
- Bates, S., Hastie, T., & Tibshirani, R. (2021). Cross-validation: What does it estimate and how well does it do it? arXiv preprint [arXiv:2104.00673](https://arxiv.org/abs/2104.00673).
- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(2), 281–305.
- Bischl, B., Binder, M., Lang, M., Pielok, T., Richter, J., Coors, S., Thomas, J., Ullmann, T., Becker, M., Anne-Laure Boulesteix, D., & Deng, & M. Lindauer. (2023). Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges. *WIREs Data Mining and Knowledge Discovery*, 13(2), e1484. <https://doi.org/10.1002/widm.1484>
- Box, G. E. P., Hunter, J. Stuart, & Hunter, William G. (2005). *Statistics for experimenters: Statistics For Experimenters: Design, Innovation, and discovery*. Wiley series in probability and statistics. Wiley Hoboken, 2nd edition.
- Cortez, P., Cerdeira, António, Almeida, F., Matos, T., & Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*, 47(4), 547–553.
- Cox, D. R., & Reid, N. (2000). *The theory of the design of experiments*. Chapman & Hall/CRC.
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923.
- Feurer, M., Hutter, F. (2019). Hyperparameter optimization. In F. Hutter, L. Kotthoff, & J. Vanschoren (Eds.), *Hyperparameter optimization: Automated machine learning— methods, systems, challenges* (pp. 3–33), Springer, Cham (CH). <https://library.oapen.org/bitstream/handle/20.500.12657/23012/1007149.pdf>.
- Fisher, R. A. (1937). *The design of experiments*. Oliver & Boyd.
- Fix, E., & Hodges, J. L. (1951). Discriminatory analysis. Nonparametric discrimination: Consistency properties. USAF School of Aviation Medicine, Randolph Field, 1951. <https://web.archive.org/web/20200926212807/https://apps.dtic.mil/dtic/tr/fulltext/u2/a800276.pdf>
- Frossard, J., & Renaud, O. (2021). Permutation tests for regression, ANOVA, and comparison of signals: The permuco package. *Journal of Statistical Software*, 99(15), 1–32.
- Good, P. (2013). *Permutation tests: A practical guide to resampling methods for testing hypotheses*. Springer Science & Business Media.
- Gorman, R. P., & Sejnowski, T. J. (1988). Analysis of hidden units in a layered network trained to classify sonar targets. *Neural Networks*, 1, 75–89.
- Gorman, K. B., Williams, T. D., & Fraser, W. R. (2014). Ecological sexual dimorphism and environmental variability within a community of antarctic penguins (genus *pygoscelis*). *PLoS ONE*, 9(3), e90081.

- Harrison, D., & Rubinfeld, D. L. (1978). Hedonic prices and the demand for clean air. *Journal of Environmental Economics and Management*, 5, 81–102.
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep reinforcement learning that matters. In *Proceedings of the AAAI conference on artificial intelligence*, vol. 32(1).
- Hutter, F., Hoos, H. H., & Leyton-Brown, K. (2014). An efficient approach for assessing hyperparameter importance. In *International conference on machine learning*, pp. 754–762, PMLR.
- Hutter, F., Hoos, H. H., Leyton-Brown, K., & Stützle, T. (2009). ParamILS: An automatic algorithm configuration framework. *Journal of Artificial Intelligence Research*, 36, 267–306.
- Islam, R., Henderson, P., Gomrokchi, M., Precup, D. (2017). Reproducibility of benchmarked deep reinforcement learning tasks for continuous control. arXiv preprint [arXiv:1708.04133](https://arxiv.org/abs/1708.04133).
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30(NIPS 2017).
- Kennedy, F. E. (1995). Randomization tests in econometrics. *Journal of Business & Economic Statistics*, 13(1), 85–94.
- Kim, J.-H. (2009). Estimating classification error rate: Repeated cross-validation, repeated hold-out and bootstrap. *Computational Statistics & Data Analysis*, 53(11), 3735–3745.
- Kohavi, R. (1996). Scaling up the accuracy of naive-bayes classifiers: A decision-tree hybrid. In *Proceedings of the second international conference on knowledge discovery and data mining*, KDD'96, pp. 202–207, Portland, Oregon, 1996. AAAI Press. <https://archive.ics.uci.edu/ml/datasets/Adult>.
- Lantz, B. (2019). *Machine learning with R: Expert techniques for predictive modeling*, 3rd Edition. Packt Publishing, 2019. data <https://www.kaggle.com/datasets/mirichoi0218/insurance>.
- Lujan-Moreno, G. A., Howard, P. R., Rojas, O. G., & Montgomery, D. C. (2018). Design of experiments and response surface methodology to tune machine learning hyperparameters, with a random forest case-study. *Expert Systems with Applications*, 109, 195–205.
- Luo, G. (2016). A review of automatic selection methods for machine learning algorithms and hyper-parameter values. *Network Modeling Analysis in Health Informatics and Bioinformatics*, 5, 18. <https://doi.org/10.1007/s13721-016-0125-6>
- Molinaro, A. M., Simon, R., & Pfeiffer, R. S. (2005). Prediction error estimation: A comparison of resampling methods. *Bioinformatics*, 21(15), 3301–3307.
- Montgomery, D. C. (2008). *Design and analysis of experiments* (7th ed.). Wiley.
- Probst, P., Boulesteix, A.-L., & Bischl, B. (2019). Tunability: Importance of hyperparameters of machine learning algorithms. *Journal of Machine Learning Research*, 20(53), 1–32.
- R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2020. URL <https://www.R-project.org/>.
- Stone, M. (1974). Cross-validatory choice and assessment of statistical predictions. *Journal of the royal statistical society: Series B (Methodological)*, 36(2), 111–133.
- Wheeler, B., & Torchiano, M. (2016). *lmPerm: Permutation tests for linear models*, 2016. R package version 2.1.0.
- Wright, M. N., & Ziegler, A. (2017). ranger: A fast implementation of random forests for high dimensional data in C++ and R. *Journal of Statistical Software*, 77(1), 1–17. <https://doi.org/10.18637/jss.v077.i01>
- Wu, C. F. J., & Hamada, M. S. (2011). *Experiments: Planning, analysis, and optimization* (3rd ed.). Wiley.
- Yang, L., & Shami, A. (2020). On hyperparameter optimization of machine learning algorithms: Theory and practice. *Neurocomputing*, 415, 295–316. <https://doi.org/10.1016/j.neucom.2020.07.061>. ISSN 0925-2312.
- Yu, T., & Zhu, H. (2020). Hyper-parameter optimization: A review of algorithms and applications. [arxiv:2003.05689](https://arxiv.org/abs/2003.05689).

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.